# Package 'AdaDTN'

July 24, 2026

**Type** Package

**Title** Adaptive Deep Tensor Networks for Function-on-Function Regression

**Version** 1.0.0

**Description** Functions for fitting adaptive deep tensor networks for nonlinear function-on-function regression with multiple functional and scalar predictors. The package learns predictor-specific functional bases end-to-end through differentiable quadrature and constructs interpretable coefficient surfaces on the response grid. It includes tools for model fitting, cross-validation, hyperparameter tuning, prediction, split conformal prediction bands, extraction and visualization of learned model components, training summaries, and simulation-based data generation.

**Depends** R (>= 3.5.0)

**Imports** goffda,
graphics,
plot3D,
stats,
torch,
utils

**License** GPL-3

**Encoding** UTF-8

**LazyData** true

**ByteCompile** true

**RoxygenNote** 7.1.2

## Contents

---

`AdaDTN_cv` *Cross-Validation Error for an AdaDTN Configuration*

---

## Description

Computes the subject-level $K$-fold cross-validation error for one hyperparameter configuration of the adaptive deep tensor network (AdaDTN). For each fold, the function fits `AdaDTN_fit` on the subjects outside that fold, predicts the complete response curves of the held-out subjects, and evaluates the mean squared prediction error over subjects and response-grid points. The returned scalar is the unweighted average of the fold-specific errors.

## Usage

```
AdaDTN_cv(
  resp,
  func_cov,
  scalar_cov = NULL,
  grid_in = NULL,
  grid_out = NULL,
  par,
  nfolds = 5,
  fold_id = NULL,
  device = NULL,
  verbose = FALSE
)
```

## Arguments

| | |
|---|---|
| `resp` | A numeric matrix containing the functional responses. Rows correspond to subjects and columns correspond to evaluations on the response grid. If there are $n$ subjects and $M$ response-grid points, `resp` must have dimension $n \times M$. |
| `func_cov` | A nonempty list of numeric matrices containing the functional predictors. Element `func_cov[[p]]` contains the $p$-th functional predictor, with subjects in rows and evaluations on its predictor-specific grid in columns. If predictor $p$ is observed at $G_p$ grid points, its matrix must have dimension $n \times G_p$. All matrices must have the same number of rows as `resp`. |
| `scalar_cov` | An optional numeric matrix of scalar predictors, with subjects in rows and scalar covariates in columns. It must have $n$ rows. The default `NULL` fits an AdaDTN model without scalar predictors. |
| `grid_in` | An optional list containing the observation grids of the functional predictors. Element `grid_in[[p]]` must have length equal to `ncol(func_cov[[p]])`. If `NULL`, each predictor grid is set to an equally spaced sequence from 0 to 1 with the required number of points. |

`grid_out` An optional numeric vector containing the response grid. Its length must equal `ncol(resp)`. If `NULL`, an equally spaced sequence from 0 to 1 is used.

`par` A named list specifying one AdaDTN hyperparameter configuration. It is processed by the internal function `AdaDTN_sanitize_par` and then passed to `AdaDTN_fit`. The list must supply `n_base_in`, because that argument has no default in `AdaDTN_fit`. Other omitted fitting arguments use the defaults of `AdaDTN_fit`.

Relevant components include `basis_hidden_in`, `dense_hidden`, `basis_dropout`, `dense_dropout`, `basis_activation`, `dense_activation`, `scalar_embed_dim`, `lambda_in_l1`, `lambda_in_ortho`, `lambda_surface_l2`, `l2_hidden`, `epochs`, `batch_size`, `lr`, `lr_min`, `patience`, `cal_prop`, and `val_prop`. Arguments controlling the data, grids, device, or verbosity should be supplied through their dedicated arguments rather than duplicated in `par`.

`nfolds` A positive integer giving the number of outer cross-validation folds. The default is 5. In ordinary use, it should be at least 2 and no greater than the number of subjects.

`fold_id` An optional vector of length $n$ assigning each subject to an outer fold. Its entries should be the integers from `1` through `nfolds`, with every fold represented. If `NULL`, the function constructs an approximately balanced random assignment by permuting `rep(seq_len(nfolds), length.out = n)`. Supply the same `fold_id` when different configurations must be evaluated on identical folds.

`device` The Torch computation device passed to `AdaDTN_fit`, typically `"cpu"` or `"cuda"`. If `NULL`, device selection is handled by `AdaDTN_fit`.

`verbose` Logical; if `TRUE`, the function reports the CV-MSE after each outer fold. Output generated internally by `AdaDTN_fit` remains suppressed because every fold-specific fit is called with `verbose = FALSE`.

## Details

AdaDTN models a complete functional response from $P$ functional predictors and optional scalar covariates. For functional predictor $p$, the adaptive basis layer learns normalized functions $\{\bar{\phi}_k^{(p)}\}_{k=1}^{K_p}$ and forms the numerical inner-product scores

$$c_{ik}^{(p)} = \int_{\mathcal{I}_{x_p}} X_i^{(p)}(s)\bar{\phi}_k^{(p)}(s)\, ds.$$

The tensor projection matrix $A^{(p)}$ maps these scores to the response grid, producing the predictor-specific front-end feature

$$U_i^{(p)} = \left(c_{i1}^{(p)}, \ldots, c_{iK_p}^{(p)}\right) A^{(p)}.$$

The front-end features from all functional predictors, together with the optional scalar predictors, are passed through the nonlinear dense head. Consequently, the cross-validation criterion assesses prediction by the complete AdaDTN operator, not the first-layer coefficient surfaces in isolation.

Let $\mathcal{F}_1, \ldots, \mathcal{F}_K$ denote the subject sets determined by `fold_id`, where $K$ equals `nfolds`. For fold $k$, the function uses

$$\mathcal{I}_{-k} = \{1, \ldots, n\} \setminus \mathcal{F}_k$$

as the outer fitting sample and $\mathcal{F}_k$ as the outer validation sample. All functional and scalar predictors, as well as the functional responses, are subset at the subject level. Therefore, all evaluations of a given response curve remain in the same outer fold, preserving dependence within a curve and preventing response-grid points from the same subject from being distributed across training and validation samples.

The same `grid_in` and `grid_out` are used in every fold. The fitted model is applied to the held-out functional and scalar predictors using `predict(..., interval = "none")`; hence the CV criterion concerns point prediction and does not evaluate conformal interval coverage or width.

Each outer fitting sample is passed unchanged to `AdaDTN_fit`. That function performs its own random subject-level division into internal training, validation, and calibration subsets according to `val_prop` and `cal_prop`. The internal training subjects determine the centering and scaling statistics and contribute gradient updates. The internal validation subjects determine early stopping, and the internal calibration subjects are used by `AdaDTN_fit` to store calibration residuals and a conformal quantile.

The outer held-out fold is not used in any of these operations. It is used only after fitting to compute the fold-specific prediction error. Because `interval = "none"`, the internally calculated conformal quantities do not enter the outer CV loss.

This nested behavior is important when interpreting the effective fitting sample: the neural network in an outer fold is optimized on the internal training subset of $\mathcal{I}_{-k}$, rather than on every subject in $\mathcal{I}_{-k}$. It also means that the CV score remains stochastic even when `fold_id` is fixed, because the inner split, neural-network initialization, mini-batch ordering, and dropout masks can vary between runs.

Let $n_k = |\mathcal{F}_k|$, let $M$ equal `ncol(resp)`, and let $\widehat{Y}_{i,-k}(t_m)$ denote the prediction for subject $i$ at response-grid point $t_m$, obtained from the model fitted without outer fold $k$. The function computes

$$\mathrm{CV}_k = \frac{1}{n_k M} \sum_{i \in \mathcal{F}_k} \sum_{m=1}^{M} \left\{ Y_i(t_m) - \widehat{Y}_{i,-k}(t_m) \right\}^2$$

This is the mean squared error over all subject-grid-point pairs in fold $k$. On a common equally spaced response grid, it is the discrete average corresponding, up to the constant length of the response domain, to the Riemann approximation of the integrated squared prediction error. It is therefore used as the empirical CV-MISE criterion for selecting the AdaDTN architecture and regularization parameters.

The returned value is

$$\mathrm{CV} = \frac{1}{K} \sum_{k=1}^{K} \mathrm{CV}_k \, .$$

Fold errors are averaged with equal weights. The automatically generated folds differ in size by at most one subject, so this is effectively the same as pooling the squared errors. If a user supplies substantially unequal folds, however, the returned result gives each fold equal weight rather than each subject equal weight.

No response-domain quadrature weights are used in this function. For an irregular `grid_out`, the criterion is an unweighted mean over observed grid points and is not a trapezoidal approximation to an integrated loss.

## Value

A numeric scalar equal to the arithmetic mean of the fold-specific CV-MSE values. Smaller values indicate better held-out point-prediction performance for the supplied configuration.

## References

Gurer, S., Bakicierler Sezer, G., Shang, H. L., and Beyaztas, U. (2026). *AdaDTN: End-to-End Adaptive Basis Learning for Nonlinear Function-on-Function Regression*. Manuscript.

Ramsay, J. O. and Silverman, B. W. (2005). *Functional Data Analysis*, 2nd ed. Springer.

Stone, M. (1974). Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B*, 36, 111–147.

## Examples

```
## Not run:
## Generate a training sample from the complex simulation mechanism.
n <- 120
J <- 51

dat <- simulate_AdaDTN_data(
  n = n,
  j = J,
  model = "complex"
)

## Define one AdaDTN configuration.
par <- list(
  n_base_in = 4,
  basis_hidden_in = c(32, 32),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  scalar_embed_dim = NULL,
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 100L,
  batch_size = 32,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 20,
  cal_prop = 0.20,
  val_prop = 0.10
)

## Use an explicit balanced fold assignment so that the same folds can
## be reused when comparing different configurations.
fold_id <- sample(rep(seq_len(5), length.out = n))

cv_mise <- AdaDTN_cv(
  resp = dat$y,
  func_cov = dat$x,
  scalar_cov = dat$x.scl,
  grid_in = rep(list(dat$meta$sx), length(dat$x)),
  grid_out = dat$meta$sy,
  par = par,
  nfolds = 5,
  fold_id = fold_id,
  verbose = TRUE
)

cv_mise

## End(Not run)
```

`AdaDTN_fit` *Fit an Adaptive Deep Tensor Network*

## Description

Fits an adaptive deep tensor network (AdaDTN) for nonlinear function-on-function regression with one or more functional predictors and optional scalar predictors. The function learns predictor-specific marginal basis functions, maps their quadrature scores to the response grid through trainable coefficient matrices, and passes the resulting features through a nonlinear dense network that predicts the complete response curve.

Model fitting includes a subject-level training, validation, and calibration split; training-set standardization; Adam optimization with a cosine learning rate schedule; validation-based early stopping; and computation of a pooled calibration residual quantile for subsequent prediction intervals.

## Usage

```
AdaDTN_fit(
  X_func,
  X_scalar = NULL,
  Y,
  grid_in,
  grid_out,
  n_base_in,
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 64),
  basis_dropout = 0.10,
  dense_dropout = 0.10,
  basis_activation = "selu",
  dense_activation = "relu",
  scalar_embed_dim = NULL,
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-04,
  lambda_surface_l2 = 1e-04,
  l2_hidden = 1e-04,
  epochs = 150,
  batch_size = 64,
  lr = 0.001,
  lr_min = 1e-05,
  patience = 20,
  cal_prop = 0.20,
  val_prop = 0.10,
  device = NULL,
  verbose = TRUE
)
```

## Arguments

`X_func` A nonempty list of numeric matrices containing the functional predictors. Element `X_func[[p]]` represents predictor $p$, with subjects in rows and evaluations on its predictor-specific grid in columns. If there are $n$ subjects and predictor $p$

is observed at $G_p$ points, this matrix must have dimension $n \times G_p$. All elements must have the same number of rows as `Y`.

`X_scalar` An optional numeric matrix of scalar predictors, with subjects in rows and covariates in columns. It must have $n$ rows. The default `NULL` fits the model without scalar predictors.

`Y` A numeric matrix containing the functional responses. Rows correspond to subjects and columns to evaluations on the response grid. For $M$ response-grid points, `Y` must have dimension $n \times M$.

`grid_in` A list of numeric vectors containing the observation grids of the functional predictors. Its length must equal `length(X_func)`, and `length(grid_in[[p]])` must equal `ncol(X_func[[p]])`. Grids should be finite, ordered increasingly, and contain at least two distinct points because they are used to construct trapezoidal quadrature weights. Predictor-specific grids and unequal grid sizes are permitted.

`grid_out` A finite numeric vector containing the response grid. Its length must equal `ncol(Y)`. AdaDTN predicts the response directly at these grid locations; it does not reconstruct the response from a fixed output basis.

`n_base_in` An integer vector giving the number $K_p$ of learned marginal basis functions for each functional predictor. Its length must equal `length(X_func)`. Each value should be a positive integer.

`basis_hidden_in`
An integer vector giving the widths of the hidden layers in every marginal basis micro-network. The same architecture is used for all learned basis functions. For example, `c(64, 64)` creates two hidden layers of width 64 before the scalar output defining a basis function.

`dense_hidden` A nonempty integer vector giving the widths of the dense hidden layers applied after the functional front-end and optional scalar preprocessing. At least one width is required by the current implementation.

`basis_dropout` Dropout probability used after each activated hidden layer of every basis micro-network. It should lie in $[0, 1)$.

`dense_dropout` Dropout probability used after each activated dense hidden layer. It should lie in $[0, 1)$.

`basis_activation`
Character string naming the activation function used in the basis micro-networks. Supported values are `"relu"`, `"selu"`, `"elu"`, `"tanh"`, `"sigmoid"`, `"identity"`, `"linear"`, and `"leakyrelu"`.

`dense_activation`
Character string naming the activation function used in the dense hidden network and, when requested, after the scalar embedding layer. Supported values are the same as for `basis_activation`.

`scalar_embed_dim`
Optional positive integer giving the dimension of a learned linear embedding of the scalar predictors. The embedding is followed by `dense_activation`. If `NULL`, scalar predictors are concatenated with the functional front-end features directly. This argument has no effect when `X_scalar = NULL`.

`lambda_in_l1` Nonnegative coefficient for the integrated absolute-value penalty on the learned marginal basis functions. The default zero disables this penalty.

`lambda_in_ortho`
Nonnegative coefficient for the approximate orthogonality penalty on the weighted Gram matrices of the learned marginal bases.

`lambda_surface_l2`
: Nonnegative coefficient for the mean squared penalty on the predictor-specific surface coefficient matrices.

`l2_hidden`
: Nonnegative coefficient multiplying the sum of squared weights in the dense hidden layers and final output layer. Biases, the explicit intercept function, the optional scalar embedding, and the basis micro-network parameters are not included in this particular penalty.

`epochs`
: Positive integer giving the maximum number of training epochs.

`batch_size`
: Positive integer giving the mini-batch size used during optimization and validation prediction.

`lr`
: Positive numeric value giving the initial Adam learning rate and the upper endpoint of the cosine learning rate schedule.

`lr_min`
: Positive numeric value giving the lower endpoint of the cosine learning rate schedule.

`patience`
: Positive integer giving the number of consecutive epochs without a strict improvement in validation mean squared error that triggers early stopping.

`cal_prop`
: Proportion of all subjects assigned to the calibration set. The internal splitter uses `floor(cal_prop * n)`, subject to retaining at least one subject in each of the training, validation, and calibration sets.

`val_prop`
: Proportion of the subjects remaining after the calibration split that is assigned to the validation set. Thus, `val_prop` is not a proportion of the original full sample.

`device`
: Optional character string specifying the torch device, commonly `"cpu"` or `"cuda"`. If `NULL`, CUDA is selected when available and CPU otherwise.

`verbose`
: Logical value. If `TRUE`, device selection, periodic training progress, early stopping, final split-specific errors, and the stored calibration half-width are printed.

## Details

Let $X_i^{(p)}$ denote functional predictor $p$ for subject $i$, observed at the grid $s_{p1}, \ldots, s_{pG_p}$, and let the functional response be observed at $t_1, \ldots, t_M$. For each predictor, AdaDTN learns $K_p$ marginal basis functions $\phi_{pk}$. Each basis function is represented by a micro multilayer perceptron taking one grid coordinate as input and returning one scalar.

The current implementation evaluates these networks at the values supplied in `grid_in`; it does not internally rescale the grids. The resulting column vectors are normalized individually using trapezoidal weights $w_{pg}$:

$$\bar{\phi}_{pk}(s_{pg}) = \frac{\phi_{pk}(s_{pg})}{\left\{\sum_{g=1}^{G_p} w_{pg}\phi_{pk}(s_{pg})^2 + 10^{-8}\right\}^{1/2}}.$$

This gives every learned basis vector approximately unit $L^2$ norm on its observation grid. Mutual orthogonality is encouraged separately through `lambda_in_ortho`; normalization alone does not make different basis functions orthogonal.

After training-set standardization, the score of subject $i$ on basis $k$ of predictor $p$ is the differentiable trapezoidal approximation

$$\xi_{ipk} = \sum_{g=1}^{G_p} w_{pg} X_i^{(p)}(s_{pg})\bar{\phi}_{pk}(s_{pg}).$$

The vector of $K_p$ scores is multiplied by a trainable matrix $A^{(p)} \in \mathbb{R}^{K_p \times M}$:

$$U_i^{(p)} = \boldsymbol{\xi}_{ip}^{\mathsf{T}} A^{(p)}.$$

Consequently, the predictor-specific discretized front-end coefficient surface is

$$\widehat{\beta}_p(s_{pg}, t_m) = \sum_{k=1}^{K_p} \bar{\phi}_{pk}(s_{pg}) A_{km}^{(p)}.$$

This surface is an interpretable description of the linear projection performed before the nonlinear dense head. When the dense head is nonlinear, it should not be interpreted as a complete marginal effect of the final prediction, because the head may transform and interact the front-end features.

The $M$-dimensional vectors $U_i^{(1)}, \ldots, U_i^{(P)}$ are concatenated. If scalar predictors are supplied, their standardized values are either concatenated directly or transformed by one learned linear layer followed by `dense_activation`, according to `scalar_embed_dim`.

The combined feature vector is passed through the hidden layers specified by `dense_hidden`. Each hidden layer applies a linear transformation, `dense_activation`, and dropout. A bias-free final linear layer maps the last hidden representation to $M$ output coordinates, after which a separately trainable $M$-vector is added as an explicit intercept function. The network therefore predicts the response directly on `grid_out`.

Subjects are randomly permuted and then divided into three disjoint sets. First, `floor(cal_prop * n)` subjects are assigned to calibration, with internal safeguards that reserve subjects for the other sets. From the remaining subjects, `floor(val_prop * n_remaining)` are assigned to validation, again with safeguards, and all remaining subjects are used for gradient updates.

For every functional predictor, scalar predictor, and response, each grid point or scalar column is standardized separately. If $x_{ij}$ is a value in column $j$, the transformation is

$$x_{ij}^{\mathrm{sc}} = \frac{x_{ij} - \widehat{\mu}_{j,\mathcal{T}}}{\widehat{\sigma}_{j,\mathcal{T}}}.$$

The centers and scales are estimated only from the training subjects $\mathcal{T}$ and are then applied to the validation and calibration subjects. A nonfinite or zero training standard deviation is replaced by one. This training-only construction prevents information from the validation and calibration sets from entering the standardization parameters.

For a mini-batch $\mathcal{B}$, the optimized loss is the mean squared error on the standardized response scale plus the enabled regularizers:

$$\mathcal{L}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|M} \sum_{i \in \mathcal{B}} \sum_{m=1}^{M} \left( Y_{im}^{\mathrm{sc}} - \widehat{Y}_{im}^{\mathrm{sc}} \right)^2 + \mathcal{R}_{\mathrm{front}} + \lambda_{\mathrm{hidden}} \mathcal{R}_{\mathrm{hidden}}.$$

The front-end penalty is accumulated across functional predictors. Its possible components are:

- an integrated absolute-basis penalty equal, for predictor $p$, to `lambda_in_l1` times the mean across its bases of $\sum_g w_{pg} |\bar{\phi}_{pk}(s_{pg})|$;
- an approximate orthogonality penalty. With $\Phi_p$ the matrix of normalized basis evaluations, $W_p = \mathrm{diag}(w_{p1}, \ldots, w_{pG_p})$, and $G_p = \Phi_p^{\mathsf{T}} W_p \Phi_p$, the implementation adds `lambda_in_ortho` times the mean absolute entry of $G_p - I_{K_p}$. This term is used only when $K_p > 1$;
- a coefficient-matrix penalty equal to `lambda_surface_l2` times $\mathrm{mean}\{(A^{(p)})^2\}$.

The dense penalty $\mathcal{R}_{\text{hidden}}$ is the sum of squared weights in all dense hidden layers and in the final output layer. It excludes biases, the explicit intercept function, the optional scalar embedding layer, and all basis micro-network parameters.

The recorded `train_loss` is the mean of the total mini-batch losses within an epoch. The recorded `valid_loss` is an unpenalized standardized-scale mean squared error. In both cases the implementation averages batch-level scalar losses; consequently, if the last batch is smaller than the others, it receives the same weight as a full batch in these reported epoch summaries.

Parameters are optimized by Adam. At epoch $e$, the learning rate is

$$\text{lr}_e = \text{lr}_{\min} + \frac{1}{2}(\text{lr} - \text{lr}_{\min}) \left[1 + \cos\left\{\frac{\pi(e-1)}{\max(1, \text{epochs} - 1)}\right\}\right].$$

Training subjects are randomly reshuffled at every epoch. Validation loss is evaluated after the epoch with dropout disabled. Whenever the validation loss is strictly below the best value observed so far, the model state is written to a temporary checkpoint. Training stops when `patience` consecutive epochs fail to improve that value, or when `epochs` is reached. The best checkpoint, rather than necessarily the last epoch, is restored before calibration and final error calculations.

After restoring the best validation checkpoint, the function predicts the calibration responses on the standardized response scale. It pools the absolute residuals over both calibration subjects and response-grid points:

$$e_{jm}^{\text{sc}} = \left| Y_{jm}^{\text{sc}} - \widehat{Y}_{jm}^{\text{sc}} \right|.$$

The vector produced by column-major vectorization of these residuals is stored as `cal_residuals_sc`. The value `q_hat` is exactly `stats::quantile(cal_residuals_sc, probs = 0.95, type = 8,names = FALSE)`. Thus, in this implementation, `q_hat` is fixed at the 0.95 quantile, is computed on the standardized response scale, uses Hyndman–Fan type 8 interpolation, and pools all calibration grid residuals. It is a stored 0.95-level summary. The package's `predict.AdaDTN_model` method recomputes a type 8 quantile from `cal_residuals_sc` when another value of `level` is requested.

Prediction limits based on these residuals must transform their standardized-scale half-widths consistently back to the original response scale. The package prediction method does this separately at every response grid point by multiplying the standardized quantile by the stored response scale. The pooled score targets average marginal behavior over the response grid; it is not a simultaneous whole-curve score such as a subject-level supremum residual. Moreover, the calculation in this fitting function does not replace the requested probability by a finite-sample-inflated conformal rank. Any formal coverage statement should therefore be matched to the exact score, quantile, and exchangeability conditions implemented by the corresponding prediction procedure.

## Value

An object of class `"AdaDTN_model"`, represented as a list with the following components:

`model` The fitted torch module, restored to the state having the lowest observed validation mean squared error.

`scalers` A list containing the training-set column centers and scales for each functional predictor (`func`), the optional scalar predictors (`scalar`), and the functional response (`Y`).

`history` A data frame with one row per completed epoch and columns `epoch`, `lr`, `train_loss`, and `valid_loss`.

`split` A list of integer subject indices named `train`, `valid`, and `calib`.

`q_hat` The type 8 empirical 0.95 quantile of the pooled absolute calibration residuals on the standardized response scale.

`cal_residuals_sc` A numeric vector containing the pooled absolute calibration residuals on the standardized response scale.

`device` The character string identifying the torch device used for fitting.

`grid_in` The supplied list of predictor grids.

`grid_out` The supplied response grid.

`n_base_in` The supplied numbers of learned marginal basis functions.

`mse_train` Elementwise mean squared prediction error for the training subjects on the original response scale.

`mse_valid` Elementwise mean squared prediction error for the validation subjects on the original response scale.

`mse_calib` Elementwise mean squared prediction error for the calibration subjects on the original response scale.

`config` A named list recording the fitting arguments `basis_hidden_in`, `dense_hidden`, `basis_dropout`, `dense_dropout`, `basis_activation`, `dense_activation`, `scalar_embed_dim`, `lambda_in_l1`, `lambda_in_ortho`, `lambda_surface_l2`, `l2_hidden`, `epochs`, `batch_size`, `lr`, `lr_min`, `patience`, `cal_prop`, and `val_prop`.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

test_dat <- simulate_AdaDTN_data(
  n = 100,
  j = 101,
  model = "complex"
)

s <- train_dat$meta$sx
t <- train_dat$meta$sy

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(list(s), length(train_dat$x)),
  grid_out = t,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  scalar_embed_dim = NULL,
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
```

```
  l2_hidden = 1e-4,
  epochs = 400,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 40,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

## Original-scale test predictions.
pred <- predict(
  fit,
  new_X_func = test_dat$x,
  new_X_scalar = test_dat$x.scl,
  interval = "none"
)

mean((pred - test_dat$yt)^2)

## Stored training history and split-specific errors.
tail(fit$history)
c(
  train = fit$mse_train,
  validation = fit$mse_valid,
  calibration = fit$mse_calib
)

## Prediction intervals obtained from the models calibration component.
pred_int <- predict(
  fit,
  new_X_func = test_dat$x,
  new_X_scalar = test_dat$x.scl,
  interval = "conformal",
  level = 0.95
)

## Inspect the first learned front-end coefficient surface.
plot_AdaDTN_surface(fit, predictor_idx = 1, smooth_spar = 0.75)

## End(Not run)
```

---

`AdaDTN_param_grid` *Construct an AdaDTN Hyperparameter Grid*

---

## Description

Constructs a data frame containing the Cartesian product of candidate hyperparameter values for AdaDTN. Atomic candidates can be supplied as ordinary vectors, whereas vector-valued model arguments, such as hidden-layer architectures, should be supplied as protected list columns.

## Usage

```
AdaDTN_param_grid(...)
```

## Arguments

| | |
|---|---|
| `...` | Named vectors, factors, or list-like objects containing candidate values for AdaDTN hyperparameters. Each argument becomes one column of the resulting grid. The argument names should normally match arguments of `AdaDTN_fit`, such as `n_base_in`, `dense_hidden`, `basis_dropout`, or `lr`.<br>Scalar-valued candidates can be supplied as ordinary vectors. Parameters whose individual candidates are themselves vectors or `NULL` should be supplied using `I(list(...))`; see Details and Examples. |

## Details

`AdaDTN_param_grid` is a small convenience wrapper around `expand.grid`:

```
expand.grid(..., stringsAsFactors = FALSE)
```

It generates every possible combination of the supplied candidate values. If parameter $r$ has $L_r$ candidates, the returned grid has

$$\prod_{r=1}^{R} L_r$$

rows. The first supplied parameter varies fastest, following the ordering used by `expand.grid`.

Ordinary atomic vectors represent alternative scalar values. For example,

```
basis_dropout = c(0.05, 0.10)
```

creates two candidate values and therefore contributes a factor of two to the number of grid rows. Character candidates remain character rather than being converted to factors because the wrapper fixes `stringsAsFactors = FALSE`.

Some arguments of `AdaDTN_fit` are vectors describing an entire architecture. These include `n_base_in`, `basis_hidden_in`, and `dense_hidden`. Each complete vector must occupy one cell of the parameter grid. Protect such candidates with `I(list(...))`. For example,

```
dense_hidden = I(list(c(32, 32), c(64, 32)))
```

defines two candidate dense architectures. Supplying `dense_hidden = c(32, 64)` instead would define the two scalar candidates 32 and 64, not the two-layer architecture `c(32, 64)`.

The same convention can be used for `n_base_in`. A scalar candidate, such as `6`, is replicated across the functional predictors by the internal parameter-sanitization step used during tuning. Predictor-specific candidate vectors can instead be represented explicitly, for example,

```
n_base_in = I(list(c(4, 4, 4), c(6, 6, 6)))
```

when the model contains three functional predictors.

Use `I(list(NULL))` to include the no-embedding specification for `scalar_embed_dim`. The internal tuning utilities interpret an empty or all-`NA` candidate for this argument as `NULL`.

## Value

A data frame containing one row for every combination of the supplied candidates and one column for every argument in `...`. Character columns are not converted to factors. Protected vector-valued candidates are retained as list-like columns suitable for processing by the AdaDTN tuning utilities.

## Examples

```
## Not run:
## Atomic vectors define alternative scalar values. Complete architectures
## are protected as list elements so that each occupies one grid cell.
grid_ada <- AdaDTN_param_grid(
  n_base_in = I(list(6, 8)),
  basis_hidden_in = I(list(c(64, 64))),
  dense_hidden = I(list(c(32, 32), c(64, 32))),
  basis_dropout = c(0.05, 0.10),
  dense_dropout = c(0.05, 0.10),
  basis_activation = "selu",
  dense_activation = "relu",
  scalar_embed_dim = I(list(NULL)),
  lambda_in_l1 = 0,
  lambda_in_ortho = c(1e-4, 5e-4),
  lambda_surface_l2 = c(1e-4, 5e-4),
  l2_hidden = c(1e-4, 5e-4),
  epochs = 250,
  batch_size = 64,
  lr = c(5e-4, 1e-3),
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10
)

nrow(grid_ada)
names(grid_ada)
grid_ada[1:3, c("n_base_in", "dense_hidden", "lr")]

## Evaluate the grid and optionally refit the selected configuration.
tune_ada <- AdaDTN_tune(
  grid = grid_ada,
  resp = train_dat$y,
  func_cov = train_dat$x,
  scalar_cov = train_dat$x.scl,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  nfolds = 5,
  refit = TRUE,
  verbose = TRUE
)

tune_ada$results
tune_ada$best_params
tune_ada$best_model

## End(Not run)
```

---

AdaDTN_tune *Tune an AdaDTN Model by Subject-Level Cross-Validation*

---

## Description

Evaluates a data frame of AdaDTN hyperparameter configurations by subject-level $K$-fold cross-validation, selects the configuration having the smallest average cross-validation mean squared error, and optionally fits an AdaDTN model using the selected configuration.

The same outer fold assignment is used for every candidate configuration so that candidates are compared on identical held-out subjects.

## Usage

```
AdaDTN_tune(
  grid,
  resp,
  func_cov,
  scalar_cov = NULL,
  grid_in = NULL,
  grid_out = NULL,
  nfolds = 5,
  device = NULL,
  refit = TRUE,
  verbose = TRUE
)
```

## Arguments

| | |
|---|---|
| `grid` | A nonempty data frame containing the candidate hyperparameter configurations. Each row represents one configuration and each column represents one argument to `AdaDTN_fit`. It is normally constructed with `AdaDTN_param_grid`.<br>The grid must contain `n_base_in`, because that argument has no default in `AdaDTN_fit`. Vector-valued candidates such as `n_base_in`, `basis_hidden_in`, and `dense_hidden` should be stored as protected list columns; see `AdaDTN_param_grid`. |
| `resp` | A numeric matrix containing the functional responses. Rows correspond to subjects and columns to evaluations on the response grid. If there are $n$ subjects and $M$ response-grid points, `resp` must have dimension $n \times M$. |
| `func_cov` | A nonempty list of numeric matrices containing the functional predictors. Element `func_cov[[p]]` contains predictor $p$, with subjects in rows and evaluations on its predictor-specific grid in columns. If predictor $p$ is observed at $G_p$ points, its matrix must have dimension $n \times G_p$. All predictor matrices must have the same number and ordering of subjects as `resp`. |
| `scalar_cov` | An optional numeric matrix of scalar predictors, with subjects in rows and scalar covariates in columns. It must have $n$ rows. The default `NULL` tunes models without scalar predictors. |
| `grid_in` | An optional list containing the observation grids of the functional predictors. Element `grid_in[[p]]` must have length `ncol(func_cov[[p]])`. If `NULL`, an equally spaced sequence from 0 to 1 having the required length is created separately for each functional predictor. |
| `grid_out` | An optional numeric vector containing the response grid. Its length must equal `ncol(resp)`. If `NULL`, an equally spaced sequence from 0 to 1 is used. |
| `nfolds` | Integer giving the number of subject-level cross-validation folds. It should be at least two and no larger than the number of subjects. Fold sizes differ by at most one under the internally generated assignment. |

| | |
|---|---|
| device | Optional character string specifying the torch device, commonly "cpu" or "cuda". If NULL, device selection is delegated to AdaDTN_fit for every fitted model. |
| refit | Logical value. If TRUE, the selected configuration is passed to AdaDTN_fit together with the complete data supplied to this function. If FALSE, selection results are returned without fitting this final model. |
| verbose | Logical value. If TRUE, the cross-validation mean squared error is printed after each configuration. When refit = TRUE, this value is also passed to the final call to AdaDTN_fit. Fold-level messages from the internal AdaDTN_cv calls are suppressed. |

## Details

The input grid must be a data frame with at least one row. For row $g$, the function extracts every column and applies the internal AdaDTN_flatten1 utility. This removes the single layer of list wrapping commonly introduced by list columns in a parameter grid. Before each model is fitted, AdaDTN_cv applies the internal AdaDTN_sanitize_par utility, which performs the type conversions required by AdaDTN_fit.

In particular, a scalar n_base_in candidate is replicated across the $P$ functional predictors, whereas a predictor-specific candidate must have length $P$. Hidden-layer widths are converted to integer vectors, optimization counts are converted to integers, continuous hyperparameters are converted to numeric values, and an empty or all-NA scalar_embed_dim candidate is interpreted as NULL.

Columns of grid should contain only arguments intended for AdaDTN_fit. The data, grids, device, and verbosity are supplied through the dedicated arguments of AdaDTN_tune. Including names such as X_func, Y, grid_in, grid_out, device, or verbose in grid can create duplicated formal arguments in the subsequent do.call.

One fold vector is generated before the configurations are evaluated:

```
fold_id <- sample(rep(seq_len(nfolds), length.out = n))
```

This construction assigns every subject to one fold and produces folds whose sizes differ by at most one. The same realized fold_id is supplied to AdaDTN_cv for every grid row. Consequently, differences among the recorded cross-validation criteria are not caused by candidates being evaluated on different outer held-out subjects.

Only the outer folds are held fixed across candidates. Each call to AdaDTN_fit remains stochastic: its outer-fold training data are randomly divided internally into gradient-training, early-stopping validation, and calibration subsets; network parameters are initialized randomly; training subjects are reshuffled by epoch; and dropout may be active. Thus, common outer folds provide a paired data comparison but do not eliminate optimization or internal-split variability.

For configuration $g$ and fold $k$, let $\mathcal{F}_k$ denote the held-out subjects. An AdaDTN_fit model is trained using subjects outside $\mathcal{F}_k$, and point predictions are obtained for the complete response curves in $\mathcal{F}_k$. The fold error is

$$\mathrm{MSE}_{gk} = \frac{1}{|\mathcal{F}_k| M} \sum_{i \in \mathcal{F}_k} \sum_{m=1}^{M} \left\{ Y_i(t_m) - \widehat{Y}_{i,-k,g}(t_m) \right\}^2 .$$

The error used for selection is

$$\mathrm{CV}_g = \frac{1}{K} \sum_{k=1}^{K} \mathrm{MSE}_{gk}.$$

This is an unweighted average of the fold-specific errors. Each fold therefore receives equal weight even when fold sizes differ by one. Within a fold, subjects and response-grid points receive equal

weight. The code does not apply response-grid quadrature weights, so the criterion is exactly a gridwise mean squared error. On a regular response grid over a normalized domain, it may also be viewed as a discrete approximation to an average integrated squared prediction error.

The outer held-out fold is not used by its fitted model. However, `AdaDTN_fit` further partitions the outer training portion: only its internal training subset contributes gradient updates, its internal validation subset determines early stopping, and its internal calibration subset is retained for calibration. Because `AdaDTN_cv` requests only point predictions, the calibration quantity produced inside each fold fit does not enter the cross-validation MSE directly.

After all configurations are evaluated, the selected row is

$$\widehat{g} = \underset{1 \le g \le G}{\arg\min}\, \mathrm{CV}_g,$$

The implementation uses `which.min`; therefore, if several finite configurations have exactly the same smallest value, the first such row is selected. The corresponding row is flattened and sanitized once more and is returned as `best_params`.

This rule estimates predictive performance over the finite candidate set represented by `grid`. It does not establish that the selected configuration is globally optimal over all possible architectures and regularization values. Searching a larger grid can reduce approximation error in the candidate set, but it also increases computation and the possibility of selection optimism.

When `refit = TRUE`, the sanitized selected parameters and the complete `resp`, `func_cov`, and `scalar_cov` supplied to `AdaDTN_tune` are passed to a new call to `AdaDTN_fit`. This is a fresh stochastic fit; it does not average fold-specific models, reuse their weights, reuse `fold_id`, or warm-start from the best cross-validation fit.

Passing the complete data to `AdaDTN_fit` does not mean that every subject is used for gradient updates. As documented for `AdaDTN_fit`, the final call creates a new subject-level training/validation/calibration split according to the selected `cal_prop` and `val_prop`. Gradient updates use the resulting training subset, validation subjects control early stopping, and calibration subjects produce the stored calibration residuals.

When `refit = FALSE`, `best_model` is `NULL`. This option is useful when only the selected configuration is required or when refitting will be performed later on a separate dataset.

## Value

An unclassed list with the following components:

`results` A data frame containing the original parameter grid with a leading `CV_MSE` column. `CV_MSE[g]` is the unweighted average of the fold-specific gridwise mean squared errors for configuration $g$. Rows retain the ordering of `grid`.

`best_row` Integer giving the row of `results` having the smallest `CV_MSE`. In the event of an exact tie, this is the first minimizing row.

`best_params` A named list containing the flattened and sanitized parameters from the selected grid row. For example, a scalar `n_base_in` is returned after replication to the number of functional predictors.

`best_model` If `refit = TRUE`, the fitted object of class `"AdaDTN_model"` returned by the final call to `AdaDTN_fit`. If `refit = FALSE`, `NULL`.

`fold_id` An integer vector of length $n$ recording the common outer fold assignment used for all configurations. Values are in `seq_len(nfolds)`.

## Examples

```
## Not run:
train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

grid_ada <- AdaDTN_param_grid(
  n_base_in = I(list(4, 6)),
  basis_hidden_in = I(list(c(32, 32))),
  dense_hidden = I(list(c(32, 32), c(64, 32))),
  basis_dropout = c(0.05, 0.10),
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  scalar_embed_dim = I(list(NULL)),
  lambda_in_l1 = 0,
  lambda_in_ortho = c(1e-4, 5e-4),
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = c(5e-4, 1e-3),
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10
)

tune_ada <- AdaDTN_tune(
  grid = grid_ada,
  resp = train_dat$y,
  func_cov = train_dat$x,
  scalar_cov = train_dat$x.scl,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  nfolds = 5,
  refit = TRUE,
  verbose = TRUE
)

tune_ada$results
tune_ada$best_row
tune_ada$best_params
tune_ada$best_model

## Point predictions from the selected and refitted model.
test_dat <- simulate_AdaDTN_data(
  n = 100,
  j = 101,
  model = "complex"
)
```

```
pred <- predict(
  tune_ada$best_model,
  new_X_func = test_dat$x,
  new_X_scalar = test_dat$x.scl,
  interval = "none"
)

mean((pred - test_dat$yt)^2)

## To keep tuning independent of the calibration sample used by the final
## fit, select parameters on an independent pilot sample.
pilot_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

pilot_tune <- AdaDTN_tune(
  grid = grid_ada,
  resp = pilot_dat$y,
  func_cov = pilot_dat$x,
  scalar_cov = pilot_dat$x.scl,
  grid_in = rep(
    list(pilot_dat$meta$sx),
    length(pilot_dat$x)
  ),
  grid_out = pilot_dat$meta$sy,
  nfolds = 5,
  refit = FALSE,
  verbose = TRUE
)

final_dat <- simulate_AdaDTN_data(
  n = 300,
  j = 101,
  model = "complex"
)

final_fit <- do.call(
  AdaDTN_fit,
  c(
    list(
      X_func = final_dat$x,
      X_scalar = final_dat$x.scl,
      Y = final_dat$y,
      grid_in = rep(
        list(final_dat$meta$sx),
        length(final_dat$x)
      ),
      grid_out = final_dat$meta$sy,
      verbose = TRUE
    ),
    pilot_tune$best_params
  )
)
```

```
## End(Not run)
```

---

`get_AdaDTN_features` *Extract Predictor-Specific AdaDTN Front-End Features*

---

## Description

Extracts the predictor-specific tensor-projection features produced by the adaptive functional front-end of a fitted AdaDTN model. For every functional predictor, the function standardizes the supplied curves using the fitted training scalers, computes their learned adaptive-basis scores, and maps those scores to the response grid through the fitted surface coefficient matrix.

## Usage

```
get_AdaDTN_features(object, X_func, X_scalar = NULL)
```

## Arguments

| | |
|---|---|
| `object` | A fitted object of class `"AdaDTN_model"`, normally returned by `AdaDTN_fit` or as the `best_model` component of `AdaDTN_tune`. |
| `X_func` | A list of numeric matrices containing the functional predictors for the subjects whose front-end features are required. Element `X_func[[p]]` must correspond to the same predictor, observation grid, and column ordering used to fit `object`. Rows are subjects and columns are functional evaluations. If there are $n_*$ supplied subjects and predictor $p$ was observed at $G_p$ grid points, the matrix must have dimension $n_* \times G_p$. |
| `X_scalar` | An optional numeric matrix containing scalar predictors for the same subjects, with subjects in rows and scalar covariates in the same column ordering used for model fitting. Although scalar values do not enter the predictor-specific functional features returned by this function, they must be supplied when `object` was fitted with scalar predictors because the implementation executes a complete forward pass to populate the model's feature cache. If `object` was fitted without scalar predictors, this argument is ignored. |

## Details

The function first transforms every supplied functional predictor using the column-specific centers and scales stored in `object$scalers$func`. For predictor $p$, grid point $g$, and new subject $i$, the transformation is

$$X_{ip}^{\mathrm{sc}}(s_{pg}) = \frac{X_{ip}(s_{pg}) - \widehat{\mu}_{pg,\mathcal{T}}}{\widehat{\sigma}_{pg,\mathcal{T}}}.$$

The center and scale were estimated from the internal training subjects during `AdaDTN_fit`. They are not re-estimated from `X_func`. Consequently, features extracted for training and new subjects are expressed in the same fitted coordinate system.

If scalar predictors are supplied and the fitted object contains scalar scalers, the scalar columns are standardized in the same way. These standardized scalar values are passed to the complete network forward pass, but they do not change the functional front-end quantities $U_i^{(p)}$ defined below.

Let $\Phi_p$ be the $G_p \times K_p$ matrix of learned and individually normalized marginal basis evaluations for predictor $p$, and let $W_p$ be the diagonal matrix of trapezoidal quadrature weights constructed from `object$grid_in[[p]]`. Its entries are

$$(\Phi_p)_{gk} = \bar{\phi}_{pk}(s_{pg}), \qquad W_p = \mathrm{diag}(w_{p1}, \ldots, w_{pG_p}).$$

For the matrix $X^{\mathrm{sc}}_{p,*}$ of standardized new curves, the fitted model computes the adaptive-basis score matrix

$$\Xi_{p,*} = X^{\mathrm{sc}}_{p,*} W_p \Phi_p,$$

Thus, its $(i, k)$ entry is the trapezoidal approximation

$$\xi_{ipk} = \sum_{g=1}^{G_p} w_{pg} X^{\mathrm{sc}}_{ip}(s_{pg}) \bar{\phi}_{pk}(s_{pg}).$$

These are the same scores used internally by the fitted AdaDTN network; the function does not refit or modify the learned basis functions.

For predictor $p$, let $A^{(p)} \in \mathbb{R}^{K_p \times M}$ denote the fitted surface coefficient matrix, where $M$ is the number of response-grid points. The returned feature matrix is

$$U_{p,*} = \Xi_{p,*} A^{(p)} \in \mathbb{R}^{n_* \times M}.$$

Equivalently, with the fitted discretized front-end coefficient surface

$$\widehat{B}_p = \Phi_p A^{(p)},$$

the features satisfy

$$U_{p,*} = X^{\mathrm{sc}}_{p,*} W_p \widehat{B}_p.$$

Therefore, `get_AdaDTN_features` returns the response-grid-indexed functional projection of each standardized input curve through its fitted front-end surface. This identity links the extracted features to the unsmoothed surfaces returned by `get_AdaDTN_surface` when surface smoothing is disabled, and to the scores returned by `get_AdaDTN_scores`.

If there are $P$ functional predictors, AdaDTN concatenates the $M$-dimensional feature rows $U_i^{(1)}, \ldots, U_i^{(P)}$. Standardized scalar predictors, or their learned scalar embedding, are then appended. The resulting vector is the input to the nonlinear dense head:

$$h_i^{(0)} = \left[ U_i^{(1)} \,\|\, \cdots \,\|\, U_i^{(P)} \,\|\, Z_i^{\mathrm{feat}} \right], \qquad \widehat{Y}_i = f_{\mathrm{dense}}(h_i^{(0)}) + \widehat{\beta}_0.$$

Accordingly, each returned matrix is a predictor-specific *pre-head feature representation*. With a nonlinear dense head, `U_list[[p]]` is not an additive contribution of predictor $p$ to the final response prediction. The dense layers may rescale, transform, and interact features from different functional and scalar predictors. A causal or marginal-effect interpretation is therefore not justified merely from the extracted feature values.

The entries of $U_i^{(p)}$ also remain on the internal latent scale learned from standardized inputs. They are not response curves on the original measurement scale, and the response inverse-standardization operation used for final predictions should not be applied to each feature matrix separately.

## Value

A named list of length `length(X_func)`. Element `p` is a numeric matrix with one row per supplied subject and one column per response-grid point. If $n_*$ subjects are supplied and `length(object$grid_out)` = `M`, each matrix has dimension $n_* \times M$.

The list elements are named `"predictor_1"`, `"predictor_2"`, and so forth. Element `p` contains $U_{p,*} = \Xi_{p,*} A^{(p)}$ for functional predictor $p$.

## References

Gurer, S., Bakicierler Sezer, G., Shang, H. L., and Beyaztas, U. *AdaDTN: End-to-End Adaptive Basis Learning for Nonlinear Function-on-Function Regression*. Manuscript.

Ramsay, J. O. and Silverman, B. W. (2005). *Functional Data Analysis*, second edition. Springer.

## Examples

```
## Not run:
train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

test_dat <- simulate_AdaDTN_data(
  n = 100,
  j = 101,
  model = "complex"
)

features <- get_AdaDTN_features(
  object = fit,
  X_func = test_dat$x,
  X_scalar = test_dat$x.scl
)

names(features)
```

```
length(features)
lapply(features, dim)

## Concatenated functional part of the dense-head input.
functional_head_input <- do.call(cbind, features)
dim(functional_head_input)

## Front-end feature curve for the first subject and first predictor.
plot(
  fit$grid_out,
  features$predictor_1[1, ],
  type = "l",
  xlab = "t",
  ylab = "Front-end feature"
)

## Compare the score and tensor-projection dimensions.
scores <- get_AdaDTN_scores(
  object = fit,
  X_func = test_dat$x,
  X_scalar = test_dat$x.scl
)

lapply(scores, dim)
lapply(features, dim)

## End(Not run)
```

---

`get_AdaDTN_input_basis` *Extract Learned AdaDTN Input Basis Functions*

---

## Description

Extracts the learned, predictor-specific marginal basis functions from the adaptive basis layer of a fitted AdaDTN model. The functions are evaluated on the predictor grid used for model fitting and are returned after the same trapezoidal $L^2$ normalization used in every AdaDTN forward pass.

## Usage

```
get_AdaDTN_input_basis(object, predictor_idx = 1L)
```

## Arguments

| | |
|---|---|
| `object` | A fitted object of class `"AdaDTN_model"`, normally returned by `AdaDTN_fit` or as the `best_model` component of `AdaDTN_tune`. |
| `predictor_idx` | A single integer identifying the functional predictor whose learned basis functions are to be extracted. Predictors are indexed in the same order as `X_func` supplied to `AdaDTN_fit`. The default is the first predictor. The value is coerced with `as.integer` and must lie between 1 and `length(object$grid_in)`. |

## Details

For functional predictor $p$, AdaDTN represents each of its $K_p$ marginal basis functions by a separate micro multilayer perceptron,

$$\widetilde{\phi}_{pk}(s) = f_{pk}(s; \Theta_{pk}), \qquad k = 1, \ldots, K_p.$$

The input is one predictor-domain coordinate and the output is one scalar basis value. Every hidden layer of a basis micro-network applies a learned linear map, the package's layer-normalization operation, the activation specified by `basis_activation`, and dropout with probability `basis_dropout`. Its final layer is linear and scalar-valued.

The basis parameters are not estimated in a separate preprocessing step. They are learned jointly with the tensor coefficient matrices, nonlinear dense head, and response output layer by backpropagating the complete AdaDTN training objective. The extracted functions are therefore response-adaptive: their learned directions depend on the functional response and on the other functional and scalar predictors represented in the fitted model.

The current implementation supplies the numerical values in `object$grid_in[[p]]` directly to the micro-networks. It does not internally rescale the predictor grid. Hence the coordinate system and spacing used during fitting are part of the learned representation.

Let $s_{p1}, \ldots, s_{pG_p}$ denote the fitted grid for predictor $p$. Before normalization, the micro-networks produce the matrix

$$\widetilde{\Phi}_p = \left[\widetilde{\boldsymbol{\phi}}_{p1}, \ldots, \widetilde{\boldsymbol{\phi}}_{pK_p}\right] \in \mathbb{R}^{G_p \times K_p},$$

where column $k$ contains $\widetilde{\phi}_{pk}(s_{pg})$ for $g = 1, \ldots, G_p$.

`get_AdaDTN_input_basis` evaluates the functions only at this stored grid. Although a trained micro-network is a mapping of a scalar coordinate, this extraction function does not accept an alternative or finer grid.

The fitted predictor grid determines trapezoidal quadrature weights. For an increasing grid, these are

$$w_{p1} = \frac{s_{p2} - s_{p1}}{2}, \qquad w_{pG_p} = \frac{s_{pG_p} - s_{p,G_p-1}}{2},$$

and, for an interior grid point,

$$w_{pg} = \frac{(s_{pg} - s_{p,g-1}) + (s_{p,g+1} - s_{pg})}{2}, \qquad g = 2, \ldots, G_p - 1.$$

Thus, irregular but increasingly ordered grids are accommodated through their actual local spacings.

Every raw basis column is normalized separately:

$$\bar{\phi}_{pk}(s_{pg}) = \frac{\widetilde{\phi}_{pk}(s_{pg})}{\left\{\sum_{r=1}^{G_p} w_{pr}\widetilde{\phi}_{pk}(s_{pr})^2 + 10^{-8}\right\}^{1/2}}.$$

The matrix returned by the function is

$$\Phi_p = \left[\bar{\boldsymbol{\phi}}_{p1}, \ldots, \bar{\boldsymbol{\phi}}_{pK_p}\right].$$

Apart from the numerical-stability constant, every column has weighted squared norm close to one:

$$\bar{\boldsymbol{\phi}}_{pk}^{\mathsf{T}} W_p \bar{\boldsymbol{\phi}}_{pk} \approx 1, \qquad W_p = \mathrm{diag}(w_{p1}, \ldots, w_{pG_p}).$$

The small constant prevents division by zero when an unnormalized basis vector is extremely close to zero.

The extraction step normalizes each column individually; it does not apply a Gram–Schmidt procedure, eigendecomposition, or matrix whitening. Mutual orthogonality is assessed through the weighted Gram matrix

$$G_p = \Phi_p^{\mathsf{T}} W_p \Phi_p.$$

During model fitting, `lambda_in_ortho` multiplies the mean absolute entry of $G_p - I_{K_p}$ when $K_p > 1$. This penalty encourages, but does not impose, weighted orthogonality. Consequently, the off-diagonal entries of the empirical Gram matrix returned from a fitted model need not be exactly zero.

When `lambda_in_l1` is positive, training also penalizes the mean trapezoidal integral of the absolute basis values. That penalty may encourage localized or sparse-looking basis functions, but extraction applies no additional thresholding or smoothing.

For standardized input curve $X_i^{(p),\mathrm{sc}}$, the extracted basis matrix defines the adaptive score vector

$$\boldsymbol{\xi}_{ip}^{\mathsf{T}} = X_i^{(p),\mathrm{sc}} W_p \Phi_p.$$

With the fitted coefficient matrix $A^{(p)} \in \mathbb{R}^{K_p \times M}$, the predictor-specific front-end feature is

$$U_i^{(p)} = \boldsymbol{\xi}_{ip}^{\mathsf{T}} A^{(p)}.$$

The associated discretized front-end coefficient surface is

$$\widehat{B}_p = \Phi_p A^{(p)}, \qquad \widehat{\beta}_p(s_{pg}, t_m) = \sum_{k=1}^{K_p} \bar{\phi}_{pk}(s_{pg}) A_{km}^{(p)}.$$

Thus, the basis functions determine how the standardized input curve is projected before the tensor coefficient matrix transports those scores to the response grid.

## Value

A numeric matrix with `length(object$grid_in[[predictor_idx]])` rows and `object$n_base_in[predictor_idx]` columns. Row $g$ contains the learned normalized basis evaluations at `object$grid_in[[predictor_idx]][g]`, and column $k$ contains the $k$-th learned basis function.

Columns are named `"basis_1"`, `"basis_2"`, and so forth. Rows are named `"s1"`, `"s2"`, and so forth. These row names are positional labels only; the actual numerical coordinates are stored in `object$grid_in[[predictor_idx]]`.

## References

Gurer, S., Bakicierler Sezer, G., Shang, H. L., and Beyaztas, U. *AdaDTN: End-to-End Adaptive Basis Learning for Nonlinear Function-on-Function Regression*. Manuscript.

Ramsay, J. O. and Silverman, B. W. (2005). *Functional Data Analysis*, second edition. Springer.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)
```

```
fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

## Extract the learned basis system for the first functional predictor.
basis_1 <- get_AdaDTN_input_basis(
  object = fit,
  predictor_idx = 1
)

dim(basis_1)
head(basis_1)

## Plot against the actual stored predictor grid, not the positional row names.
s <- fit$grid_in[[1]]
matplot(
  s,
  basis_1,
  type = "l",
  lty = 1,
  xlab = "s",
  ylab = "Learned basis value"
)
legend(
  "topright",
  legend = colnames(basis_1),
  col = seq_len(ncol(basis_1)),
  lty = 1,
  bty = "n"
)
```

```
## Reconstruct the weighted Gram matrix used by the orthogonality penalty.
d <- diff(s)
w <- numeric(length(s))
w[1] <- d[1] / 2
w[length(s)] <- d[length(d)] / 2
if (length(s) > 2) {
  w[2:(length(s) - 1)] <-
    (d[-length(d)] + d[-1]) / 2
}

gram_1 <- crossprod(basis_1, basis_1 * w)
diag(gram_1)
gram_1

## Extract all predictor-specific basis systems.
all_bases <- lapply(
  seq_along(fit$grid_in),
  function(p) get_AdaDTN_input_basis(fit, predictor_idx = p)
)

lapply(all_bases, dim)

## End(Not run)
```

---

`get_AdaDTN_intercept` *Extract the AdaDTN Intercept Function*

---

## Description

Extracts the explicit response-grid intercept function from a fitted AdaDTN model and transforms it from the internally standardized response scale to the original response scale. The result is a named numeric vector evaluated at the response grid used for model fitting.

## Usage

```
get_AdaDTN_intercept(object)
```

## Arguments

| | |
|---|---|
| `object` | A fitted object of class `"AdaDTN_model"`, normally returned by `AdaDTN_fit` or as the `best_model` component of `AdaDTN_tune`. |

## Details

AdaDTN predicts the response directly at the $M$ locations in `object$grid_out`. Its final output layer is bias-free, and the model contains a separately parameterized vector

$$\boldsymbol{\beta}_0^{\mathrm{sc}} = \{\beta_0^{\mathrm{sc}}(t_1), \ldots, \beta_0^{\mathrm{sc}}(t_M)\}^{\mathsf{T}} \in \mathbb{R}^M.$$

This vector is stored as `object$model$intercept_fun`. It is shared by all subjects but may vary freely over the response grid, so it is a functional intercept rather than a single scalar intercept.

For subject $i$, let $r_i^{\mathrm{sc}}$ denote the standardized-scale output of the bias-free final linear layer after the adaptive functional front-end, optional scalar processing, and nonlinear dense hidden layers. The network's standardized prediction is

$$\widehat{\boldsymbol{Y}}_i^{\mathrm{sc}} = \boldsymbol{r}_i^{\mathrm{sc}} + \boldsymbol{\beta}_0^{\mathrm{sc}}.$$

The explicit intercept therefore provides one unrestricted baseline coordinate for every response-grid location and is added only after the subject-specific dense-head output has been computed.

During fitting, every response-grid column is standardized using its center and standard deviation estimated from the internal training subjects. Let

$$\widehat{\boldsymbol{\mu}}_Y = (\widehat{\mu}_{Y,1}, \ldots, \widehat{\mu}_{Y,M})^{\mathsf{T}}, \qquad \widehat{\boldsymbol{\sigma}}_Y = (\widehat{\sigma}_{Y,1}, \ldots, \widehat{\sigma}_{Y,M})^{\mathsf{T}}$$

denote `object$scalers$Y$center` and `object$scalers$Y$scale`, respectively. Full response predictions are returned to the original scale pointwise:

$$\widehat{Y}_i(t_m) = \widehat{\mu}_{Y,m} + \widehat{\sigma}_{Y,m} \left\{ r_i^{\mathrm{sc}}(t_m) + \beta_0^{\mathrm{sc}}(t_m) \right\}.$$

Rearranging this expression gives

$$\widehat{Y}_i(t_m) = \underbrace{\left\{ \widehat{\mu}_{Y,m} + \widehat{\sigma}_{Y,m} \beta_0^{\mathrm{sc}}(t_m) \right\}}_{\widehat{\beta}_0(t_m)} + \widehat{\sigma}_{Y,m} r_i^{\mathrm{sc}}(t_m).$$

Accordingly, `get_AdaDTN_intercept` returns exactly

$$\widehat{\boldsymbol{\beta}}_0 = \widehat{\boldsymbol{\mu}}_Y + \widehat{\boldsymbol{\sigma}}_Y \odot \boldsymbol{\beta}_0^{\mathrm{sc}},$$

where $\odot$ denotes elementwise multiplication. The operation is performed separately at every response-grid point because the response was standardized column by column.

If the learned standardized intercept is identically zero, the returned function is the training response-center curve $\widehat{\boldsymbol{\mu}}_Y$, not a zero vector. This follows from expressing the explicit intercept on the original response scale.

The returned vector is the original-scale version of the model's *explicit* intercept parameter. It should not automatically be interpreted as:

- the mean observed response curve;
- the conditional mean response when the predictors equal their training means;
- the prediction for a hypothetical subject with zero standardized functional and scalar predictors; or
- a predictor-specific effect.

Although the final output layer has no bias, the dense hidden layers have their own bias vectors. The optional scalar embedding also has a bias when it is present. Thus, even when all standardized functional and scalar inputs are zero, the nonlinear head can generate a nonzero vector $\boldsymbol{r}_0^{\mathrm{sc}}$. The prediction at the training-center inputs is then

$$\widehat{\boldsymbol{Y}}_{\mathrm{center}} = \widehat{\boldsymbol{\beta}}_0 + \widehat{\boldsymbol{\sigma}}_Y \odot \boldsymbol{r}_0^{\mathrm{sc}},$$

which need not equal the value returned by this function.

More generally, a flexible biased hidden network can represent constant components as well as predictor-dependent components. Some constant signal may therefore be allocated to the explicit intercept and some to the dense head. The explicit intercept is a clearly identified model parameter in the stored architecture, but its numerical decomposition from constant hidden-head output need not have a unique scientific interpretation.

## Value

A named numeric vector of length `length(object$grid_out)`. Element $m$ equals

$$\widehat{\beta}_0(t_m) = \widehat{\mu}_{Y,m} + \widehat{\sigma}_{Y,m}\beta_0^{\mathrm{sc}}(t_m).$$

The elements are named `"t1"`, `"t2"`, and so forth. These names are positional labels only; the corresponding numerical response-grid coordinates are stored in `object$grid_out`.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

beta0 <- get_AdaDTN_intercept(fit)

length(beta0)
head(beta0)

## Plot against the actual response grid.
t <- fit$grid_out
plot(
  t,
```

```
  beta0,
  type = "l",
  lwd = 2,
  xlab = "t",
  ylab = expression(hat(beta)[0](t))
)

## Compare the fitted explicit intercept with the response center computed
## from the models internal gradient-training subjects.
response_center <- fit$scalers$Y$center

lines(
  t,
  response_center,
  col = 2,
  lty = 2,
  lwd = 2
)

legend(
  "topright",
  legend = c("Explicit intercept", "Training response center"),
  col = c(1, 2),
  lty = c(1, 2),
  lwd = 2,
  bty = "n"
)

## Prediction at training-center inputs. This need not equal beta0 because
## hidden-layer biases can produce an additional constant contribution.
mean_X_func <- lapply(
  fit$scalers$func,
  function(sc) matrix(sc$center, nrow = 1)
)

mean_X_scalar <- matrix(
  fit$scalers$scalar$center,
  nrow = 1
)

pred_at_center <- predict(
  fit,
  new_X_func = mean_X_func,
  new_X_scalar = mean_X_scalar,
  interval = "none"
)

head(
  cbind(
    explicit_intercept = beta0,
    prediction_at_center = as.numeric(pred_at_center)
  )
)

## End(Not run)
```

`get_AdaDTN_scores` *Extract AdaDTN Adaptive-Basis Scores*

## Description

Extracts subject-specific projection scores for the learned marginal basis functions of every functional predictor in a fitted AdaDTN model. Supplied curves are standardized using the fitted training scalers and integrated against the learned, normalized basis functions by the same trapezoidal quadrature rule used in the model's forward pass.

## Usage

```
get_AdaDTN_scores(object, X_func, X_scalar = NULL)
```

## Arguments

| | |
|---|---|
| `object` | A fitted object of class `"AdaDTN_model"`, normally returned by `AdaDTN_fit` or as the `best_model` component of `AdaDTN_tune`. |
| `X_func` | A list of numeric matrices containing the functional predictors for the subjects whose adaptive-basis scores are required. Element `X_func[[p]]` must correspond to the same predictor, observation grid, and column ordering used to fit `object`. Rows are subjects and columns are functional evaluations. If there are $n_*$ supplied subjects and predictor $p$ was observed at $G_p$ grid points, its matrix must have dimension $n_* \times G_p$. |
| `X_scalar` | An optional numeric matrix containing scalar predictors for the same subjects, with subjects in rows and scalar covariates in the column order used for model fitting. Scalar values do not enter the adaptive-basis score formula directly. They must nevertheless be supplied when `object` was fitted with scalar predictors because the current implementation executes a complete forward pass to populate the model's score cache. If `object` was fitted without scalar predictors, this argument is ignored. |

## Details

Each supplied functional predictor is transformed using the column-specific centers and scales stored in `object$scalers$func`. For predictor $p$, subject $i$, and grid point $g$,

$$X_{ip}^{\mathrm{sc}}(s_{pg}) = \frac{X_{ip}(s_{pg}) - \widehat{\mu}_{pg,\mathcal{T}}}{\widehat{\sigma}_{pg,\mathcal{T}}}.$$

The centers and scales were estimated using only the internal training subjects in `AdaDTN_fit`. They are reused without modification, so training, validation, calibration, and new-subject scores share the same fitted coordinate system.

This pointwise standardization is important for interpretation. The returned scores are projections of the *standardized* curves, not the curves in their original measurement units. A grid location with large raw variability is therefore not automatically given more influence solely because of its measurement scale.

When both `X_scalar` and the fitted scalar scaler are present, scalar columns are standardized in the same way. The standardized scalar matrix is needed only to complete the model forward pass; at fixed fitted parameters, it does not change the values stored in the functional score cache.

For functional predictor $p$, let

$$\Phi_p = \left[\bar{\phi}_{p1}, \ldots, \bar{\phi}_{pK_p}\right] \in \mathbb{R}^{G_p \times K_p}$$

be the learned basis matrix evaluated on `object$grid_in[[p]]`. Column $k$ is produced by its own basis micro-network and is normalized using the predictor-specific trapezoidal weights:

$$\bar{\phi}_{pk}(s_{pg}) = \frac{\widetilde{\phi}_{pk}(s_{pg})}{\left\{\sum_{r=1}^{G_p} w_{pr}\widetilde{\phi}_{pk}(s_{pr})^2 + 10^{-8}\right\}^{1/2}}.$$

The basis functions are learned jointly with the complete prediction network; they are not fixed B-spline, Fourier, or functional principal-component functions chosen before seeing the response.

Normalization makes the weighted squared norm of each basis column close to one. It does not force distinct columns to be exactly orthogonal. Approximate orthogonality is encouraged during fitting by the `lambda_in_ortho` penalty on the weighted Gram matrix $\Phi_p^{\mathsf{T}} W_p \Phi_p$.

Let

$$W_p = \mathrm{diag}(w_{p1}, \ldots, w_{pG_p})$$

be the trapezoidal weight matrix constructed from the stored predictor grid. For all supplied subjects, the function returns the matrix

$$\Xi_{p,*} = X^{\mathrm{sc}}_{p,*} W_p \Phi_p \in \mathbb{R}^{n_* \times K_p}.$$

Its $(i, k)$ entry is

$$\xi_{ipk} = \sum_{g=1}^{G_p} w_{pg} X^{\mathrm{sc}}_{ip}(s_{pg}) \bar{\phi}_{pk}(s_{pg}).$$

This is the trapezoidal approximation to the functional inner product between the standardized predictor curve and the learned marginal basis function. For an irregular increasing grid, the weights reflect the actual adjacent grid spacings.

The score map is linear in a new standardized functional predictor once the fitted basis functions are held fixed. The complete AdaDTN regression operator is nevertheless nonlinear because the basis system was learned jointly from the response and the resulting score-derived features are processed by a nonlinear dense head.

## Value

A named list with one element for every functional predictor. For predictor $p$, element `p` is a numeric matrix with one row per supplied subject and `object$n_base_in[p]` columns. If $n_*$ subjects are supplied, its dimension is $n_* \times K_p$.

List elements are named `"predictor_1"`, `"predictor_2"`, and so forth. The returned matrices receive no explicit row or column names; column $k$ corresponds to `"basis_k"` in the matrix returned by `get_AdaDTN_input_basis` for the same predictor.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)
```

```
fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

test_dat <- simulate_AdaDTN_data(
  n = 100,
  j = 101,
  model = "complex"
)

scores <- get_AdaDTN_scores(
  object = fit,
  X_func = test_dat$x,
  X_scalar = test_dat$x.scl
)

names(scores)
length(scores)
lapply(scores, dim)
head(scores$predictor_1)

## Verify the quadrature formula for the first predictor.
s <- fit$grid_in[[1]]
d <- diff(s)
w <- numeric(length(s))
w[1] <- d[1] / 2
w[length(s)] <- d[length(d)] / 2
if (length(s) > 2) {
  w[2:(length(s) - 1)] <-
    (d[-length(d)] + d[-1]) / 2
```

```
}

X1_sc <- sweep(
  sweep(
    test_dat$x[[1]],
    2,
    fit$scalers$func[[1]]$center,
    "-"
  ),
  2,
  fit$scalers$func[[1]]$scale,
  "/"
)

Phi_1 <- get_AdaDTN_input_basis(
  fit,
  predictor_idx = 1
)

scores_1_manual <- sweep(X1_sc, 2, w, "*")
max(abs(scores_1_manual - scores$predictor_1))

## Verify the next tensor-projection step for the first predictor.
A_1 <- as.matrix(
  fit$model$get_A(1)$detach()$cpu()
)

features <- get_AdaDTN_features(
  object = fit,
  X_func = test_dat$x,
  X_scalar = test_dat$x.scl
)

features_1_from_scores <- scores$predictor_1
max(abs(features_1_from_scores - features$predictor_1))

## End(Not run)
```

---

`get_AdaDTN_surface` *Extract an AdaDTN Front-End Coefficient Surface*

---

## Description

Extracts the predictor-specific front-end coefficient surface from a fitted AdaDTN model. The exact fitted surface is the product of the learned normalized input-basis matrix and its trainable tensor coefficient matrix. An optional two-directional spline smoother can be applied to the extracted matrix for visualization.

## Usage

```
get_AdaDTN_surface(
  object,
  predictor_idx = 1,
  smooth_spar = 0.5
)
```

## Arguments

| | |
|---|---|
| `object` | A fitted object of class `"AdaDTN_model"`, normally returned by `AdaDTN_fit` or as the `best_model` component of `AdaDTN_tune`. |
| `predictor_idx` | A single integer identifying the functional predictor whose front-end surface is to be extracted. Predictors are indexed in the same order as `X_func` supplied to `AdaDTN_fit`. The default is the first predictor. The value is coerced with `as.integer` and must lie between 1 and `length(object$grid_in)`. |
| `smooth_spar` | Smoothing parameter supplied to `smooth.spline` during optional post-estimation smoothing. The same value is used in the response- and predictor-grid directions. The default is `0.5`. Set this argument to `NULL`, zero, or a negative value to return the exact unsmoothed fitted surface. A positive value is passed directly as `spar`; it should be a single finite numeric value accepted by `smooth.spline`. |

## Details

For functional predictor $p$, let

$$\Phi_p = \left[\bar{\phi}_{p1}, \ldots, \bar{\phi}_{pK_p}\right] \in \mathbb{R}^{G_p \times K_p}$$

denote the learned marginal basis functions evaluated on `object$grid_in[[p]]`. Every column is normalized using the predictor-specific trapezoidal weights. Let

$$A^{(p)} \in \mathbb{R}^{K_p \times M}$$

be the fitted tensor coefficient matrix, where $M$ is the number of locations in `object$grid_out`. Before optional smoothing, the function computes

$$\widehat{B}_p = \Phi_p A^{(p)} \in \mathbb{R}^{G_p \times M}.$$

Its entry at predictor-grid point $s_{pg}$ and response-grid point $t_m$ is

$$\widehat{\beta}_p(s_{pg}, t_m) = \sum_{k=1}^{K_p} \bar{\phi}_{pk}(s_{pg}) A^{(p)}_{km}.$$

The factorization is low-rank in the predictor-domain direction:

$$\operatorname{rank}(\widehat{B}_p) \le \min(K_p, G_p, M).$$

Unlike an approach that expands the response in a prespecified basis, AdaDTN estimates one free coefficient column for every response-grid location through $A^{(p)}$. The response is subsequently predicted directly on `grid_out`.

For a standardized input curve, AdaDTN computes the adaptive-basis score row

$$\boldsymbol{\xi}_{ip}^{\mathsf{T}} = X_i^{(p),\mathrm{sc}} W_p \Phi_p,$$

where $W_p$ is the diagonal matrix of trapezoidal predictor-grid weights. The predictor-specific feature sent toward the nonlinear head is

$$U_i^{(p)} = \boldsymbol{\xi}_{ip}^{\mathsf{T}} A^{(p)} = X_i^{(p),\mathrm{sc}} W_p \widehat{B}_p.$$

Thus, the exact unsmoothed matrix returned with `smooth_spar = NULL` or a nonpositive value is the same surface used inside the fitted network to transform predictor $p$. The scores and features in these identities can be extracted with `get_AdaDTN_scores` and `get_AdaDTN_features`, respectively.

The surface describes the learned linear functional projection placed at the front of the network. At response-grid index $m$, it determines how the standardized values of predictor $p$ are integrated to construct the $m$-th coordinate of $U_i^{(p)}$. It is therefore a directly inspectable diagnostic of where and how the adaptive front-end extracts response-relevant information.

In the general AdaDTN model, the vectors $U_i^{(1)}, \ldots, U_i^{(P)}$ are concatenated with the scalar features and passed through nonlinear dense layers. Consequently, $\widehat{B}_p$ is not generally the derivative of the final response prediction with respect to predictor $p$, nor is it an additive predictor contribution on the original response scale.

To see this distinction, let $f_{\text{dense}}$ denote the subject-specific mapping from the concatenated front-end features to the standardized response prediction, and let $J_{ip}$ be its Jacobian with respect to $U_i^{(p)}$. A perturbation of the standardized predictor at grid location $s_{pg}$ changes the front-end feature row by

$$\frac{\partial U_i^{(p)}}{\partial X_i^{(p),\text{sc}}(s_{pg})} = w_{pg}\widehat{B}_p[g, \cdot].$$

The corresponding final standardized-response derivative involves the additional factor $J_{ip}$ and may depend on the complete subject profile:

$$\frac{\partial \widehat{\boldsymbol{Y}}_i^{\text{sc}}}{\partial X_i^{(p),\text{sc}}(s_{pg})} = J_{ip}\left\{w_{pg}\widehat{B}_p[g, \cdot]\right\}^{\mathsf{T}}.$$

The extracted surface alone is therefore a pre-nonlinearity coefficient surface.

In the special linear-head construction in which predictor-specific $U_i^{(p)}$ vectors are added directly to the response and no nonlinear interactions are introduced, the surface reduces to the familiar discretized function-on-function coefficient surface on the internally standardized scale.

## Value

A numeric matrix with `length(object$grid_in[[predictor_idx]])` rows and `length(object$grid_out)` columns. Row $g$ corresponds to `object$grid_in[[predictor_idx]][g]`, and column $m$ corresponds to `object$grid_out[m]`.

If `smooth_spar` is `NULL` or nonpositive, the matrix is exactly $\Phi_p A^{(p)}$ up to numerical conversion from torch. If `smooth_spar` is positive, the returned value is the sequentially smoothed version described above.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
```

```
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

## Exact fitted front-end surface for predictor 1.
surface_raw <- get_AdaDTN_surface(
  object = fit,
  predictor_idx = 1,
  smooth_spar = NULL
)

## Post-estimation smoothed version used for visual presentation.
surface_smooth <- get_AdaDTN_surface(
  object = fit,
  predictor_idx = 1,
  smooth_spar = 0.75
)

dim(surface_raw)
dim(surface_smooth)
max(abs(surface_raw - surface_smooth))

## Verify the exact fitted factorization.
Phi_1 <- get_AdaDTN_input_basis(
  fit,
  predictor_idx = 1
)

A_1 <- as.matrix(
  fit$model$get_A(1)$detach()$cpu()
)

surface_manual <- Phi_1
max(abs(surface_raw - surface_manual))

## Display the exact surface using the actual predictor and response grids.
s <- fit$grid_in[[1]]
t <- fit$grid_out
```

```
persp(
  x = s,
  y = t,
  z = surface_raw,
  theta = 40,
  phi = 20,
  expand = 0.5,
  xlab = "s",
  ylab = "t",
  zlab = "Front-end surface"
)

## The package plotting utility can apply the same optional smoothing.
plot_AdaDTN_surface(
  fit,
  predictor_idx = 1,
  smooth_spar = 0.75
)

## End(Not run)
```

---

`plot.AdaDTN_model` *Plot Diagnostics and Learned Components of an AdaDTN Model*

---

## Description

Provides the S3 plotting method for fitted AdaDTN models. Depending on `type`, it displays a predictor-specific front-end coefficient surface, the learned marginal basis functions for one functional predictor, or the training and validation loss histories.

## Usage

```
## S3 method for class AdaDTN_model
plot(
  x,
  type = c("surface", "basis", "history"),
  predictor_idx = 1,
  ...
)
```

## Arguments

| | |
|---|---|
| `x` | A fitted object of class `"AdaDTN_model"`, normally returned by `AdaDTN_fit` or as the `best_model` component of `AdaDTN_tune`. |
| `type` | Character string selecting the plot. Available choices are `"surface"`, `"basis"`, and `"history"`. The default is `"surface"`. Partial matching is performed by `match.arg`. |
| `predictor_idx` | A single integer identifying the functional predictor used when `type = "surface"` or `type = "basis"`. Predictors are indexed in the same order as `X_func` supplied to `AdaDTN_fit`. The argument is ignored when `type = "history"`. |

`...` Additional arguments whose destination depends on `type`. For `"surface"`, they are passed to `plot_AdaDTN_surface` and may include its named arguments such as `smooth_spar`, `view_theta`, `view_phi`, `surface_col`, `border_col`, and `shade_fac`. For `"basis"` and `"history"`, they are passed to `matplot`.

Arguments already fixed explicitly by the selected branch should not be repeated in `...`, because duplicated formal arguments can generate an error. See Details.

## Details

Objects returned by `AdaDTN_fit` inherit from `"AdaDTN_model"`. Calling `plot(x)` dispatches to this method and, because the first value of `type` is `"surface"`, produces the front-end surface plot for the first functional predictor by default.

The method applies `match.arg(type)` before plotting. Hence an unambiguous abbreviation such as `type = "hist"` is accepted, whereas an invalid or ambiguous value raises an error.

The method is a plotting and extraction interface only. It does not refit the network, update parameters, recompute training losses, alter calibration residuals, or modify the stored data split.

With `type = "surface"`, the method calls

```
plot_AdaDTN_surface(
  object = x,
  predictor_idx = predictor_idx,
  ...
)
```

and immediately returns its result. That function obtains the predictor-specific front-end surface from `get_AdaDTN_surface` and draws it with `persp3D`. The package **plot3D** must therefore be installed for this mode.

For predictor $p$, the exact fitted surface is

$$\widehat{B}_p = \Phi_p A^{(p)}, \qquad \widehat{\beta}_p(s_{pg}, t_m) = \sum_{k=1}^{K_p} \bar{\phi}_{pk}(s_{pg}) A^{(p)}_{km}.$$

It describes the linear tensor projection placed before the nonlinear dense head. With a nonlinear head, the displayed surface is not generally the final marginal derivative or an additive contribution on the original response scale.

Unless overridden through `...`, the delegated plotting function uses `smooth_spar = 0.5`. It therefore displays a sequentially spline-smoothed version of the exact fitted matrix. Smoothing is a post-estimation graphical operation and does not change model predictions. Set `smooth_spar = NULL` or a nonpositive value to display the exact matrix $\Phi_p A^{(p)}$. Full details of the index-based smoothing and surface scale are provided in `get_AdaDTN_surface`.

Named surface arguments should use the interface of `plot_AdaDTN_surface`; for example, use `surface_col` rather than passing a second `col` argument through its final `....` Arguments that duplicate values already fixed in `plot_AdaDTN_surface`, such as `x`, `y`, `z`, or `zlim`, can be matched more than once in the eventual `persp3D` call and cause an error.

With `type = "basis"`, the method first calls `get_AdaDTN_input_basis` for the selected predictor. If $G_p$ is its number of grid points and $K_p$ its number of adaptive basis functions, the extracted matrix is

$$\Phi_p = \left[\bar{\phi}_{p1}, \ldots, \bar{\phi}_{pK_p}\right] \in \mathbb{R}^{G_p \times K_p}.$$

The columns are plotted against the actual numerical predictor grid in `x$grid_in[[predictor_idx]]`. The branch fixes `type = "l"`, `lty = 1`, the axis labels, and the title `"Adaptive bases: predictor`

p". Additional nonconflicting graphical arguments, such as col, lwd, ylim, or log, can be supplied through ....

The displayed basis functions are the individually trapezoidally normalized functions used by the fitted model. They are not additionally smoothed for plotting. The orthogonality penalty used during fitting encourages, but does not enforce, zero weighted inner products between distinct columns.

Individual basis columns remain subject to sign, permutation, and rotation ambiguities. Their shapes show the response-adaptive directions learned by the model, but a particular numbered basis should not be assigned an intrinsic meaning across independently trained fits. The learned subspace and the rotation-invariant surface are more stable interpretive targets.

Do not repeat type, lty, xlab, ylab, or main in ... for this branch, because these are already supplied explicitly to matplot.

With type = "history", the method plots x$history$train_loss and x$history$valid_loss against x$history$epoch. It uses solid lines with colors "#1B6CA8" and "#C73E1D", respectively, and adds a top-right legend labeled "Training" and "Validation".

The two curves have related but not identical definitions:

- train_loss is the within-epoch mean of the mini-batch training objectives. Each mini-batch objective contains standardized-scale response MSE, enabled adaptive-basis and surface penalties, and l2_hidden times the dense/output weight penalty.
- valid_loss is an unpenalized standardized-response MSE, evaluated with the network in evaluation mode.

Accordingly, the vertical separation between the training and validation curves is not a pure generalization gap: the training curve includes regularization terms and is evaluated with training-mode dropout, whereas the validation curve excludes the penalties and disables dropout.

Both stored quantities are averages of batch-level scalar losses. If the last batch is smaller, it receives the same weight as a full batch in the reported epoch summary. Neither curve is expressed on the original response scale.

The history contains one row for every completed epoch. Early stopping is triggered after x$config$patience consecutive epochs without a strict improvement in validation loss, and the fitted model restores the checkpoint having the smallest observed validation loss. The plot does not mark that best epoch automatically. If training continued through later nonimproving epochs before stopping, the parameters stored in x$model correspond to the best checkpoint rather than necessarily to the final row in the history.

The branch fixes type, lty, col, xlab, ylab, and main in its matplot call. These arguments should not be duplicated in .... Other nonconflicting arguments such as ylim, xlim, lwd, or log may be supplied. The separately drawn legend always uses the fixed colors and line types shown above and does not receive ....

## Value

The method is called primarily for its graphical side effect. Its invisible return value depends on type:

"surface" The value returned by plot_AdaDTN_surface: an invisible list with components s, t, and surface. The first two are the predictor and response grids, and surface is the displayed matrix after any requested smoothing.

"basis" The numeric basis matrix returned by get_AdaDTN_input_basis, invisibly. It has one row per selected predictor-grid point and one column per learned basis function.

"history" The x$history data frame, invisibly, with columns epoch, lr, train_loss, and valid_loss.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

## Default S3 plot: smoothed surface for the first predictor.
surface_info <- plot(fit)

## Exact unsmoothed surface with a different viewing angle.
surface_exact <- plot(
  fit,
  type = "surface",
  predictor_idx = 1,
  smooth_spar = NULL,
  view_theta = 50,
  view_phi = 15
)

names(surface_exact)
dim(surface_exact$surface)

## Learned adaptive basis functions for the second predictor.
```

```
basis_values <- plot(
  fit,
  type = "basis",
  predictor_idx = 2,
  col = seq_len(fit$n_base_in[2]),
  lwd = 2
)

dim(basis_values)

## Penalized training loss and unpenalized validation MSE.
history_values <- plot(
  fit,
  type = "history",
  lwd = 2
)

head(history_values)

## Identify the epoch whose validation loss supplied the restored checkpoint.
best_epoch <- history_values$epoch[
  which.min(history_values$valid_loss)
]
best_epoch

## End(Not run)
```

---

`plot_AdaDTN_surface` *Plot an AdaDTN Front-End Coefficient Surface*

---

## Description

Draws a three-dimensional perspective plot of the predictor-specific front-end coefficient surface from a fitted AdaDTN model. The plotted surface may be the exact fitted tensor product or an optionally smoothed post-estimation version.

## Usage

```
plot_AdaDTN_surface(
  object,
  predictor_idx = 1,
  smooth_spar = 0.75,
  view_theta = 40,
  view_phi = 2,
  surface_col = "#8EC9F3",
  border_col = "#666666",
  shade_fac = 0.15,
  ...
)
```

## Arguments

| | |
|---|---|
| `object` | A fitted object of class `"AdaDTN_model"`, normally returned by `AdaDTN_fit` or as the `best_model` component of `AdaDTN_tune`. |

| | |
|---|---|
| predictor_idx | A single integer identifying the functional predictor whose front-end surface is plotted. Predictors are indexed in the same order as X_func supplied to AdaDTN_fit. The default is the first predictor. |
| smooth_spar | Smoothing parameter passed to get_AdaDTN_surface. The default 0.75 applies sequential spline smoothing in the response- and predictor-index directions before plotting. Set this argument to NULL, zero, or a negative value to plot the exact fitted surface $\Phi_p A^{(p)}$. |
| view_theta | Numeric azimuthal viewing angle, in degrees, passed as theta to persp3D. The default is 40. |
| view_phi | Numeric colatitude or elevation viewing angle, in degrees, passed as phi to persp3D. The default is 2. |
| surface_col | Color specification passed as col to persp3D for the surface facets. The default is the light blue color "#8EC9F3". |
| border_col | Color specification passed as border to persp3D for the facet borders. The default is "#666666". |
| shade_fac | Numeric shading value passed as shade to persp3D. The default is 0.15. |
| ... | Additional graphical arguments passed to persp3D. Arguments already supplied explicitly by this function must not be repeated in ...; duplicated formal arguments can cause an error. See Details. |

## Details

For the selected functional predictor $p$, the exact AdaDTN front-end surface is

$$\widehat{B}_p = \Phi_p A^{(p)} \in \mathbb{R}^{G_p \times M},$$

where $\Phi_p$ contains the learned, trapezoidally normalized marginal basis functions and $A^{(p)} \in \mathbb{R}^{K_p \times M}$. Its entries are

$$\widehat{\beta}_p(s_{pg}, t_m) = \sum_{k=1}^{K_p} \bar{\phi}_{pk}(s_{pg}) A^{(p)}_{km}.$$

The function delegates extraction to get_AdaDTN_surface. Therefore, the matrix plotted is:

- the exact fitted surface when smooth_spar is NULL or nonpositive; or
- a sequentially spline-smoothed version when smooth_spar is positive.

The default is the second case. Smoothing is performed first across the response-grid column indices for each predictor-grid row and then across the predictor-grid row indices for each response-grid column. The spline coordinates are integer indices rather than the numerical values of the grids. Full details are given in get_AdaDTN_surface.

Smoothing is applied only to the matrix returned for display. It does not alter the torch module, coefficient matrices, learned bases, tensor- projection features, predictions, training history, or calibration quantities.

The horizontal plotting coordinates are taken directly from the fitted object:

$$x_g = \text{grid\_in}_p[g], \qquad y_m = \text{grid\_out}[m],$$

These vectors are coerced to numeric. The extracted $G_p \times M$ matrix is supplied as z, so its rows correspond to predictor-grid values $s$ and its columns correspond to response-grid values $t$.

view_theta controls horizontal rotation around the surface, while view_phi controls the vertical viewing angle. These values affect only the rendering and do not change the returned surface matrix.

The function calls persp3D with the following settings fixed by its implementation:

- facet line width `lwd = 0.5`;
- vertical expansion `expand = 0.5`;
- axis labels `xlab = "s"` and `ylab = "t"`;
- an empty vertical-axis label, `zlab = ""`;
- detailed ticks, `ticktype = "detailed"`;
- enlarged tick and axis-label sizes, `cex.axis = 2` and `cex.lab = 2`;
- vertical plotting limits `zlim = c(-0.3, 0.4)`; and
- `main = NULL`, because the title is added separately.

The facet color, border color, shading, and viewing angles are controlled by the dedicated arguments of this function.

The fixed `zlim` affects the display only. Surface entries outside `c(-0.3, 0.4)` remain present in the invisibly returned matrix but may be clipped or omitted from the visible plotting range. Because `zlim` is already supplied explicitly, passing another `zlim` through `...` results in duplicate matching rather than overriding it. Changing the vertical limit therefore requires changing this plotting function or drawing the matrix returned by `get_AdaDTN_surface` with a separate plotting call.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
```

```
  verbose = TRUE
)

## Default display: post-estimation smoothing with spar = 0.75.
displayed <- plot_AdaDTN_surface(
  object = fit,
  predictor_idx = 1
)

names(displayed)
dim(displayed$surface)
range(displayed$surface)

## Exact fitted tensor-product surface without graphical smoothing.
exact <- plot_AdaDTN_surface(
  object = fit,
  predictor_idx = 1,
  smooth_spar = NULL,
  view_theta = 50,
  view_phi = 15,
  surface_col = "#76B7E5",
  border_col = "#555555",
  shade_fac = 0.20
)

## Confirm that the returned matrix is the exact extracted surface.
surface_raw <- get_AdaDTN_surface(
  fit,
  predictor_idx = 1,
  smooth_spar = NULL
)

max(abs(exact$surface - surface_raw))

## Plot another functional predictor.
plot_AdaDTN_surface(
  object = fit,
  predictor_idx = 2,
  smooth_spar = 0.5,
  view_theta = 35,
  view_phi = 10,
  surface_col = "#A8DADC"
)

## End(Not run)
```

---

`predict.AdaDTN_model` *Predict Functional Responses from an AdaDTN Model*

---

## Description

Provides the S3 prediction method for fitted AdaDTN models. It standardizes new functional and scalar predictors using the fitted training scalers, predicts complete response curves on the original response scale, and can optionally construct symmetric calibration-based prediction bands from the stored pooled standardized residuals.

## Usage

```
## S3 method for class AdaDTN_model
predict(
  object,
  new_X_func,
  new_X_scalar = NULL,
  interval = c("none", "conformal"),
  level = 0.95,
  batch_size = 256,
  ...
)
```

## Arguments

| | |
|---|---|
| `object` | A fitted object of class `"AdaDTN_model"`, normally returned by `AdaDTN_fit` or as the `best_model` component of `AdaDTN_tune`. |
| `new_X_func` | A list of numeric matrices containing the functional predictors for the new subjects. Element `new_X_func[[p]]` must correspond to the same predictor, observation grid, and column order used to fit `object`. Rows are subjects and columns are functional evaluations. If there are $n_*$ new subjects and predictor $p$ was observed at $G_p$ grid points, its matrix must have dimension $n_* \times G_p$. All list elements must contain the same subjects in the same row order. |
| `new_X_scalar` | An optional numeric matrix of scalar predictors for the new subjects, with subjects in rows and covariates in the same column order used for model fitting. It must be supplied when `object` was fitted with scalar predictors and must be `NULL` when the fitted model contains no scalar predictors. |
| `interval` | Character string specifying the returned result. Use `"none"` for point predictions only or `"conformal"` for point predictions and symmetric calibration-based lower and upper bands. The default is `"none"`. Partial matching is performed by `match.arg`. |
| `level` | Numeric probability used as the empirical quantile level when `interval = "conformal"`. The default is `0.95`. It should be a single finite value between zero and one. This argument is ignored when `interval = "none"`. |
| `batch_size` | Positive integer giving the maximum number of new subjects processed in one torch forward pass. The default is 256. This controls prediction memory use but does not change the fitted parameters. |
| `...` | Additional arguments included for compatibility with the `predict` generic. They are not used by the current method. |

## Details

Every functional predictor is transformed using the column-specific centers and scales stored in `object$scalers$func`. For new subject $i$, functional predictor $p$, and predictor-grid location $g$,

$$X^{\mathrm{sc}}_{ip,*}(s_{pg}) = \frac{X_{ip,*}(s_{pg}) - \widehat{\mu}_{X,pg,\mathcal{T}}}{\widehat{\sigma}_{X,pg,\mathcal{T}}}.$$

The centers and scales were estimated only from the internal gradient- training subjects in `AdaDTN_fit`. They are not re-estimated from the new data.

When scalar predictors are present, their columns are transformed analogously using `object$scalers$scalar`. If the fitted model expects scalar predictors but `new_X_scalar = NULL`, the model's forward method

stops because its scalar input is missing. Conversely, supplying `new_X_scalar` to a model fitted without scalar predictors attempts to use a nonexistent scalar scaler and is not supported.

The method does not accept predictor grids as arguments. It assumes that every column of `new_X_func[[p]]` corresponds exactly to the stored coordinate `object$grid_in[[p]][g]`. It performs no interpolation, registration, or grid matching.

For predictor $p$, the standardized new functional curve is projected onto the learned normalized basis system:

$$\boldsymbol{\xi}_{ip}^{\mathsf{T}} = X_{ip,*}^{\mathrm{sc}} W_p \Phi_p,$$

The score vector is mapped to the response grid by the fitted coefficient matrix:

$$U_i^{(p)} = \boldsymbol{\xi}_{ip}^{\mathsf{T}} A^{(p)}.$$

All predictor-specific feature vectors are concatenated with the standardized scalar features or their learned embedding and passed through the fitted nonlinear dense head. A bias-free output layer and the explicit functional intercept produce the standardized response prediction

$$\widehat{\boldsymbol{Y}}_{i,*}^{\mathrm{sc}} = f_{\widehat{\Theta}}\left(X_{i,*}^{(1:P),\mathrm{sc}}, Z_{i,*}^{\mathrm{sc}}\right) \in \mathbb{R}^M.$$

The network predicts directly at the locations in `object$grid_out`; no fixed output-basis reconstruction is performed.

The fitted response center and scale are then applied pointwise:

$$\widehat{Y}_{i,*}(t_m) = \widehat{\mu}_{Y,m,\mathcal{T}} + \widehat{\sigma}_{Y,m,\mathcal{T}} \widehat{Y}_{i,*}^{\mathrm{sc}}(t_m).$$

The returned point predictions are therefore on the original response scale.

## Value

The return value depends on `interval`:

`"none"` A numeric matrix of point predictions on the original response scale. It has one row per new subject and `length(object$grid_out)` columns.

`"conformal"` A list with three numeric matrices:

`mean` Point predictions on the original response scale.

`lower` Lower symmetric calibration-based limits on the original response scale.

`upper` Upper symmetric calibration-based limits on the original response scale.

Each matrix has one row per new subject and one column per stored response- grid location. No additional result class is assigned.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 300,
  j = 101,
  model = "complex"
)

test_dat <- simulate_AdaDTN_data(
  n = 100,
  j = 101,
```

```
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  scalar_embed_dim = NULL,
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 400,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 40,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

## Original-scale point predictions.
pred <- predict(
  fit,
  new_X_func = test_dat$x,
  new_X_scalar = test_dat$x.scl,
  interval = "none",
  batch_size = 256
)

dim(pred)
mean((pred - test_dat$yt)^2)

## Symmetric calibration-based bands at empirical quantile level 0.95.
pred_int <- predict(
  fit,
  new_X_func = test_dat$x,
  new_X_scalar = test_dat$x.scl,
  interval = "conformal",
  level = 0.95
)

names(pred_int)
lapply(pred_int, dim)
```

```
## The original-scale width is the same across subjects at each grid point.
band_width <- pred_int$upper - pred_int$lower
apply(band_width, 2, range)

## Descriptive pooled, pointwise, and whole-curve coverage on simulated
## observed test responses. Only the first is aligned with the pooled
## empirical summary; none should be treated as a guarantee from one sample.
covered <- (
  test_dat$y >= pred_int$lower &
  test_dat$y <= pred_int$upper
)

pooled_coverage <- mean(covered)
pointwise_coverage <- colMeans(covered)
whole_curve_coverage <- mean(
  rowSums(covered) == ncol(covered)
)

pooled_coverage
range(pointwise_coverage)
whole_curve_coverage

## A different level recomputes the type 8 quantile from stored residuals.
pred_int_90 <- predict(
  fit,
  new_X_func = test_dat$x,
  new_X_scalar = test_dat$x.scl,
  interval = "conformal",
  level = 0.90
)

mean(pred_int_90$upper - pred_int_90$lower)
mean(pred_int$upper - pred_int$lower)

## Verify the exact standardized quantile used at level 0.95.
q_sc_95 <- quantile(
  fit$cal_residuals_sc,
  probs = 0.95,
  type = 8,
  names = FALSE
)

all.equal(
  as.numeric(q_sc_95),
  as.numeric(fit$q_hat)
)

## End(Not run)
```

---

`print.AdaDTN_model` *Print an AdaDTN Model Summary*

---

## Description

Prints a concise console summary of a fitted AdaDTN model, including the numbers of functional predictors, response-grid points, and adaptive basis functions; the computation device; the number of completed epochs; and the final training, validation, and calibration mean squared errors.

## Usage

```
## S3 method for class AdaDTN_model
print(x, ...)
```

## Arguments

| | |
|---|---|
| x | A fitted object of class "AdaDTN_model", normally returned by AdaDTN_fit or as the best_model component of AdaDTN_tune. |
| ... | Additional arguments included for compatibility with the print generic. They are not used by the current method. |

## Value

The input object x, returned invisibly and unchanged. The method is called primarily for its console-printing side effect.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
```

```
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

## Explicit printing.
print(fit)

## Entering the object at the console invokes the same S3 method.
fit

## The fitted object is returned invisibly.
returned_fit <- print(fit)
identical(returned_fit, fit)

## Capture the fixed console display as character lines.
printed_lines <- capture.output(print(fit))
printed_lines

## Components not included in the concise display remain accessible.
fit$config
fit$q_hat

best_epoch <- fit$history$epoch[
  which.min(fit$history$valid_loss)
]

c(
  epochs_completed = nrow(fit$history),
  best_validation_epoch = best_epoch
)

## End(Not run)
```

---

`print.summary.AdaDTN_model` *Print a Detailed AdaDTN Model Summary*

---

## Description

Prints the structured summary produced by `summary.AdaDTN_model`. The display reports the fitted architecture size, subject-split sizes, optimization device and epoch information, original-scale split-specific mean squared errors, and the stored standardized calibration half-width.

## Usage

```
## S3 method for class summary.AdaDTN_model
print(x, ...)
```

## Arguments

| | |
|---|---|
| x | An object of class "summary.AdaDTN_model", normally produced by summary(fitted_model) or summary.AdaDTN_model. |
| ... | Additional arguments included for compatibility with the print generic. They are not used or forwarded by the current method. |

## Value

The input summary object x, returned invisibly and unchanged. The method is called primarily for its console-printing side effect.

## Examples

```
## Not run:

train_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 250,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 35,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

fit_summary <- summary(fit)

## Explicit printing of the summary object.
print(fit_summary)
```

```
## The summary object is returned invisibly.
returned_summary <- print(fit_summary)
identical(returned_summary, fit_summary)

## Capture the fixed display.
summary_lines <- capture.output(print(fit_summary))
summary_lines

## Programmatic access to components shown in the display.
fit_summary$split_sizes
fit_summary$mse
fit_summary$best_validation_epoch
fit_summary$q_hat

## Configuration settings are stored but not printed.
fit_summary$config

## Convert the printed standardized 0.95 half-width to the original response
## scale at every response-grid point.
q_original <- (
  fit_summary$q_hat *
  fit$scalers$Y$scale
)

head(q_original)

## End(Not run)
```

---

`simulate_AdaDTN_data` *Simulate Function-on-Function Data for AdaDTN*

---

## Description

Generates synthetic function-on-function regression data with five functional predictors, three scalar predictors, and a functional response observed on regular grids over $[0, 1]$. Three mechanisms are available: a linear baseline, a nonlinear interaction model, and a complex model with localized predictor directions and nonlinear mixed-predictor effects.

The noise-free response and the response contaminated by a rescaled Ornstein–Uhlenbeck error process are both returned.

## Usage

```
simulate_AdaDTN_data(
  n,
  j,
  model = c("linear", "nonlinear", "complex"),
  n_func = 5,
  n_scl = 3
)
```

## Arguments

| | |
|---|---|
| n | Positive integer giving the number of simulated subjects. |
| j | Integer giving the common number of predictor- and response-grid points. Both grids are constructed as `seq(0, 1, length.out = j)`. A value of at least two is required for a meaningful functional grid and for the Ornstein–Uhlenbeck simulation. |
| model | Character string selecting the response mechanism. Available choices are `"linear"`, `"nonlinear"`, and `"complex"`. The default is `"linear"`. Partial matching is performed by `match.arg`. |
| n_func | Integer giving the requested number of functional predictors. The default and intended value is 5. The supplied nonlinear and complex mechanisms are hard-coded for five functional predictors, and the complex predictor generator contains five predictor-specific location pairs. Consequently, `n_func = 5` should be retained for the documented data-generating processes; see Details. |
| n_scl | Integer giving the requested number of scalar predictors. The default and intended value is 3. The scalar response profiles and the nonlinear and complex mechanisms are explicitly written for three scalar predictors. Consequently, `n_scl = 3` should be retained for the documented data-generating processes. |

## Details

The predictor and response grids are identical regular sequences,

$$s_g = \frac{g-1}{j-1}, \qquad t_m = \frac{m-1}{j-1}, \qquad g, m = 1, \ldots, j.$$

They are returned as `meta$sx` and `meta$sy`. Although the numerical grids coincide, they represent distinct predictor and response domains.

For compactness, define the Gaussian bump

$$B(x; c, w) = \exp\left\{-\frac{1}{2}\left(\frac{x-c}{w}\right)^2\right\}$$

and the localized wave

$$W(x; c, w, f) = \sin(f\pi x) B(x; c, w).$$

These functions are implemented internally as `gauss_bump` and `smooth_wave`.

When `model` is `"linear"` or `"nonlinear"`, each functional predictor is a rank-one random curve

$$X_i^{(k)}(s_g) = \xi_{ik}\phi_k(s_g).$$

For $k \leq 3$,

$$\xi_{ik} \sim N\left(4, \frac{1}{k^2}\right), \qquad \phi_k(s) = \sin\{(k+1)\pi s\}.$$

For $k > 3$,

$$\xi_{ik} \sim N\left(0, \frac{16}{k^8}\right), \qquad \phi_k(s) = \cos\{(k-2)\pi s\}.$$

Thus, the first three functional predictors have relatively strong nonzero-mean amplitudes, whereas later predictors have centered amplitudes whose standard deviations decrease rapidly with $k$. No additional pointwise measurement noise is added to these rank-one predictors.

When `model = "complex"`, every functional predictor contains four random structured components plus independent pointwise noise. For predictor $k \in \{1, \ldots, 5\}$, independently generate

$$a_{i1}^{(k)} \sim N(0, 1^2), \quad a_{i2}^{(k)} \sim N(0, 0.9^2), \quad a_{i3}^{(k)} \sim N(0, 0.7^2), \quad a_{i4}^{(k)} \sim N(0, 0.5^2).$$

The predictor-specific centers are

$$\boldsymbol{c}_1 = (0.15, 0.28, 0.40, 0.58, 0.76), \qquad \boldsymbol{c}_2 = (0.25, 0.38, 0.55, 0.70, 0.86).$$

Using the $k$-th entries of these vectors, define

$$b_{k1}(s) = B(s; c_{1k}, 0.06), \qquad b_{k2}(s) = B(s; c_{2k}, 0.08),$$

$$b_{k3}(s) = W(s; c_{1k} + 0.10, 0.09, 5 + k),$$

and

$$b_{k4}(s) = \cos\{(4 + k)\pi s\} B(s; 0.82, 0.10).$$

The curve is

$$X_i^{(k)}(s_g) = \sum_{r=1}^{4} a_{ir}^{(k)} b_{kr}(s_g) + \delta_{ikg}, \qquad \delta_{ikg} \sim N(0, 0.03^2).$$

The resulting predictors contain localized bumps, localized oscillations, and a small pointwise-noise component. The coefficient draws and pointwise errors are generated separately for every functional predictor.

The function constructs five $j \times j$ matrices from

$$\beta_1(s, t) = \exp\{-2(s - 0.5)^2\} \sin(6\pi t),$$

$$\beta_2(s, t) = \cos(6\pi s) \exp\{-2(t - 0.5)^2\},$$

$$\beta_3(s, t) = \sin(5\pi s) \cos(5\pi t),$$

$$\beta_4(s, t) = 4st \exp\{-2(s^2 + t^2)\},$$

and

$$\beta_5(s, t) = 5\sqrt{s + 0.1} \log(t + 1).$$

The corresponding matrices are returned in `meta$beta`.

Only the first three surfaces enter the common baseline functional signal:

$$F_i(t_m) = \sum_{k=1}^{3} \frac{1}{j} \sum_{g=1}^{j} X_i^{(k)}(s_g) \beta_k(s_g, t_m).$$

The implementation uses the equal-weight factor $1/j$; it does not use trapezoidal quadrature weights in the data generator.

Although five surfaces are stored, $\beta_4$ and $\beta_5$ do not enter this baseline linear sum. In the nonlinear mechanism, predictors 4 and 5 enter through separate nonlinear terms. In the complex mechanism, all five predictors enter through localized projections described below.

With the intended setting `n_scl = 3`, the scalar predictors are independent draws

$$Z_{i\ell} \sim N(2, 1), \qquad \ell = 1, 2, 3.$$

Their response profiles are

$$\gamma_1(t) = \sin(4\pi t), \qquad \gamma_2(t) = \cos(5\pi t),$$

and

$$\gamma_3(t) = 2\sin(3\pi t)\exp(-t).$$

The scalar signal and common baseline are

$$S_i(t_m) = \sum_{\ell=1}^{3} Z_{i\ell}\gamma_\ell(t_m), \qquad H_i(t_m) = F_i(t_m) + S_i(t_m).$$

The matrix $H$ is called `base_signal` internally.

For `model = "linear"`, the noise-free response is exactly the common baseline:

$$Y_i^{\text{true}}(t_m) = H_i(t_m).$$

Only functional predictors 1–3 enter the response through the baseline surfaces, while all three scalar predictors enter through their response profiles. Functional predictors 4 and 5 are generated but do not affect the linear response.

For `model = "nonlinear"`, five nonlinear terms are added to the common baseline.

First, predictors 1 and 2 interact pointwise through

$$N_{i1}(t_m) = \frac{1}{j}\sum_{g=1}^{j} X_i^{(1)}(s_g)X_i^{(2)}(s_g)\left[12\exp\left\{-15\left((s_g-0.4)^2+(t_m-0.6)^2\right)\right\}\right].$$

Second, form the response-grid vector

$$R_{i3}(t_m) = \frac{1}{j}\sum_{g=1}^{j} X_i^{(3)}(s_g)\beta_3(s_g,t_m)$$

and average it over the response grid,

$$\bar{R}_{i3} = \frac{1}{j}\sum_{m=1}^{j} R_{i3}(t_m).$$

The second nonlinear term is

$$N_{i2}(t_m) = \sin(2\pi\bar{R}_{i3})\left[8\sin(6\pi t_m)\cos(2\pi t_m)\right].$$

Third, functional predictor 4 interacts with scalar predictor 1:

$$N_{i3}(t_m) = \frac{1}{j}\sum_{g=1}^{j}\left\{X_i^{(4)}(s_g)Z_{i1}\right\}\left[10 s_g t_m\exp(-3 s_g t_m)\right].$$

Fourth, scalar predictors 2 and 3 form

$$N_{i4}(t_m) = Z_{i2}^2 Z_{i3}\left[7\exp(-t_m)\cos(4\pi t_m)\right].$$

Fifth, functional predictor 5 interacts with scalar predictor 1:

$$N_{i5}(t_m) = \arctan\left\{Z_{i1}\frac{1}{j}\sum_{g=1}^{j} X_i^{(5)}(s_g)\right\}\left[6t_m\sin(3\pi t_m)\right].$$

Let $N = N_1 + \cdots + N_5$. The implementation calculates the root-mean-square magnitudes

$$\|H\|_{\mathrm{RMS}} = \left\{ \frac{1}{nj} \sum_{i=1}^{n} \sum_{m=1}^{j} H_i(t_m)^2 \right\}^{1/2},$$

and analogously $\|N\|_{\mathrm{RMS}}$. It rescales the complete nonlinear sum to have twice the RMS magnitude of the baseline:

$$\widetilde{N} = N \frac{2\|H\|_{\mathrm{RMS}}}{\|N\|_{\mathrm{RMS}}}.$$

The noise-free response is

$$Y^{\mathrm{true}} = H + \widetilde{N}.$$

The rescaling is applied jointly to the sum of all five nonlinear terms, not to the terms separately.

For `model = "complex"`, each of the five functional predictors is reduced through a localized direction:

$$p_{i1} = \frac{1}{j} \sum_{g=1}^{j} X_i^{(1)}(s_g) B(s_g; 0.18, 0.06),$$

$$p_{i2} = \frac{1}{j} \sum_{g=1}^{j} X_i^{(2)}(s_g) B(s_g; 0.42, 0.07),$$

$$p_{i3} = \frac{1}{j} \sum_{g=1}^{j} X_i^{(3)}(s_g) B(s_g; 0.64, 0.07),$$

$$p_{i4} = \frac{1}{j} \sum_{g=1}^{j} X_i^{(4)}(s_g) W(s_g; 0.72, 0.10, 7),$$

and

$$p_{i5} = \frac{1}{j} \sum_{g=1}^{j} X_i^{(5)}(s_g) W(s_g; 0.85, 0.08, 9).$$

The five response profiles are

$$g_1(t) = B(t; 0.16, 0.09),$$

$$g_2(t) = B(t; 0.36, 0.10) \left\{ 1 + 0.25 \sin(6\pi t) \right\},$$

$$g_3(t) = B(t; 0.58, 0.11) \cos(5\pi t),$$

$$g_4(t) = B(t; 0.76, 0.09) \sin(7\pi t),$$

and

$$g_5(t) = B(t; 0.88, 0.07) \left\{ 1 + 0.20 \cos(8\pi t) \right\}.$$

The subject-specific nonlinear coefficients are

$$z_{i1} = \tanh(2.2 p_{i1}),$$

$$z_{i2} = \tanh(1.8 p_{i2}) \tanh(1.8 p_{i3}),$$

$$z_{i3} = \sin(2.5 p_{i4} + 0.6 Z_{i1}),$$

$$z_{i4} = \tanh\left[ 1.5 \left\{ p_{i2} + 0.7 Z_{i2} \right\} \right],$$

and

$$z_{i5} = \arctan(1.8 p_{i5} + 0.5 Z_{i3}).$$

Before balancing, term $r$ is the outer product

$$C_{ir}(t_m) = z_{ir} g_r(t_m).$$

The reference standard deviation is

$$\sigma_{\mathrm{ref}} = 0.45 \,\mathrm{sd}\left\{H_i(t_m) : i = 1, \ldots, n,\ m = 1, \ldots, j\right\}.$$

Each nonlinear term is rescaled separately:

$$\widetilde{C}_r = C_r \frac{\sigma_{\mathrm{ref}}}{\mathrm{sd}\{\mathrm{vec}(C_r)\}}.$$

If a term's empirical standard deviation is zero or nonfinite, the internal `balance_term` function returns that term unchanged.

Finally,

$$Y^{\mathrm{true}} = 0.45H + \sum_{r=1}^{5} \widetilde{C}_r.$$

Because the balanced terms need not be mutually uncorrelated, this construction fixes their individual empirical standard deviations but does not fix the standard deviation of their sum.

## Value

A list with the following components:

`y` Numeric $n \times j$ matrix containing the observed functional responses $Y^{\mathrm{obs}}$. Rows are subjects and columns are response-grid points.

`yt` Numeric $n \times j$ matrix containing the noise-free functional responses $Y^{\mathrm{true}}$.

`x` List of `n_func` numeric $n \times j$ matrices containing the functional predictors.

`x.scl` Numeric $n \times \mathrm{n\_scl}$ matrix containing the scalar predictors.

`meta` A list with:

- `sx` Numeric predictor grid of length `j`.
- `sy` Numeric response grid of length `j`.
- `beta` List of five baseline $j \times j$ coefficient-surface matrices.
- `model` Character string identifying the selected mechanism.

## Examples

```
## Not run:

## Generate independent training and test samples from the complex mechanism.
train_dat <- simulate_AdaDTN_data(
  n = 300,
  j = 101,
  model = "complex"
)

test_dat <- simulate_AdaDTN_data(
  n = 100,
  j = 101,
  model = "complex"
)
```

```
str(train_dat, max.level = 2)
lapply(train_dat$x, dim)
dim(train_dat$x.scl)
dim(train_dat$y)
dim(train_dat$yt)

## Inspect the noise scale created by the empirical rescaling step.
signal_sd <- sd(as.vector(train_dat$yt))
noise_sd <- sd(as.vector(train_dat$y - train_dat$yt))
c(
  signal_sd = signal_sd,
  noise_sd = noise_sd,
  noise_to_signal_sd = noise_sd / signal_sd
)

## Display several simulated functional predictors and responses.
matplot(
  train_dat$meta$sx,
  t(train_dat$x[[1]][1:10, ]),
  type = "l",
  lty = 1,
  xlab = "s",
  ylab = expression(X^{(1)}(s))
)

matplot(
  train_dat$meta$sy,
  t(train_dat$y[1:10, ]),
  type = "l",
  lty = 1,
  xlab = "t",
  ylab = "Y(t)"
)

## Fit AdaDTN using the returned package-ready structure.
fit <- AdaDTN_fit(
  X_func = train_dat$x,
  X_scalar = train_dat$x.scl,
  Y = train_dat$y,
  grid_in = rep(
    list(train_dat$meta$sx),
    length(train_dat$x)
  ),
  grid_out = train_dat$meta$sy,
  n_base_in = rep(4, length(train_dat$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
  dense_activation = "relu",
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 400,
  batch_size = 64,
```

```
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 40,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

pred <- predict(
  fit,
  new_X_func = test_dat$x,
  new_X_scalar = test_dat$x.scl,
  interval = "none"
)

mean((pred - test_dat$yt)^2)

## Generate all three mechanisms with the intended dimensions.
linear_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "linear"
)

nonlinear_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "nonlinear"
)

complex_dat <- simulate_AdaDTN_data(
  n = 200,
  j = 101,
  model = "complex"
)

## End(Not run)
```

---

`summary.AdaDTN_model` *Summarize a Fitted AdaDTN Model*

---

## Description

Constructs a structured summary of a fitted adaptive deep tensor network (AdaDTN) model. The summary records model dimensions, subject-split sizes, optimization information, split-specific mean squared errors, the stored conformal calibration quantile, and the fitting configuration.

## Usage

```
## S3 method for class AdaDTN_model
summary(object, ...)
```

## Arguments

| | |
|---|---|
| object | An object of class "AdaDTN_model", ordinarily returned by AdaDTN_fit or stored in the best_model component returned by AdaDTN_tune. |
| ... | Additional arguments supplied for compatibility with the summary generic. They are currently ignored. |

## Value

An object of class "summary.AdaDTN_model", implemented as a list with the following components:

| | |
|---|---|
| n_functional_predictors | Number of functional predictors, equal to length(object$grid_in). |
| n_response_grid_points | Number of response-grid points, equal to length(object$grid_out). |
| n_base_in | The stored vector giving the number of adaptive input bases for each functional predictor. |
| split_sizes | A named numeric vector containing the numbers of training, validation, and calibration subjects. |
| device | The device stored in the fitted object, commonly "cpu" or "cuda". |
| epochs_completed | The number of completed epochs, equal to nrow(object$history). |
| best_validation_epoch | The epoch associated with the first minimum of object$history$valid_loss. |
| mse | A named numeric vector with elements train, validation, and calibration, containing the corresponding stored original-scale mean squared errors. |
| q_hat | The stored type-8 0.95 quantile of the pooled absolute calibration residuals on the standardized response scale. |
| config | The fitting-configuration list stored in object$config. |

## Examples

```
## Not run:

sim <- simulate_AdaDTN_data(
  n = 160,
  j = 101,
  model = "complex"
)

fit <- AdaDTN_fit(
  X_func = sim$x,
  X_scalar = sim$x.scl,
  Y = sim$y,
  grid_in = rep(list(sim$meta$sx), length(sim$x)),
  grid_out = sim$meta$sy,
  n_base_in = rep(4L, length(sim$x)),
  basis_hidden_in = c(64, 64),
  dense_hidden = c(64, 32),
  basis_dropout = 0.10,
  dense_dropout = 0.05,
  basis_activation = "tanh",
```

```
  dense_activation = "relu",
  scalar_embed_dim = NULL,
  lambda_in_l1 = 0,
  lambda_in_ortho = 1e-4,
  lambda_surface_l2 = 1e-4,
  l2_hidden = 1e-4,
  epochs = 400,
  batch_size = 64,
  lr = 1e-3,
  lr_min = 1e-5,
  patience = 40,
  cal_prop = 0.20,
  val_prop = 0.10,
  verbose = TRUE
)

fit_summary <- summary(fit)
print(fit_summary)
str(fit_summary)

fit_summary$n_functional_predictors
fit_summary$n_response_grid_points
fit_summary$n_base_in
fit_summary$split_sizes
fit_summary$device
fit_summary$epochs_completed
fit_summary$best_validation_epoch
fit_summary$mse
fit_summary$q_hat
fit_summary$config

# The reported epoch is the first epoch attaining the minimum
# stored validation loss.
best_epoch <- fit$history$epoch[
  which.min(fit$history$valid_loss)
]
identical(fit_summary$best_validation_epoch, best_epoch)

# Convert the stored standardized 0.95 half-width to the
# original response scale at every response-grid point.
half_width_95 <- fit_summary$q_hat * fit$scalers$Y$scale

inherits(fit_summary, "summary.AdaDTN_model")

## End(Not run)
```

# Index